

Beginning Java Objects

From Concepts to Code, Second Edition

JACQUIE BARKER

Apress®

www.traintelco.com

Beginning Java Objects: From Concepts to Code, Second Edition

Copyright © 2005 by Jacquie Barker

All rights reserved. No part of this work may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage or retrieval system, without the prior written permission of the copyright owner and the publisher.

ISBN (pbk): 1-59059-457-6

Printed and bound in the United States of America 9 8 7 6 5 4 3 2 1

Trademarked names may appear in this book. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

Lead Editor: Steve Anglin

Technical Reviewer: James Huddleston

Editorial Board: Steve Anglin, Dan Appleman, Ewan Buckingham, Gary Cornell, Tony Davis, Jason Gilmore, Jonathan Hassell, Chris Mills, Dominic Shakeshaft, Jim Sumser

Assistant Publisher: Grace Wong

Project Manager: Beckie Stones

Copy Edit Manager: Nicole LeClerc

Copy Editors: Nicole LeClerc, Ami Knox

Production Manager: Kari Brooks-Copony

Production Editor: Katie Stence

Compositor and Artist: Kinetic Publishing Services, LLC

Proofreader: Sue Boshers

Indexer: Broccoli Information Management

Interior Designer: Van Winkle Design Group

Cover Designer: Kurt Krames

Manufacturing Manager: Tom Debolski

Distributed to the book trade in the United States by Springer-Verlag New York, Inc., 233 Spring Street, 6th Floor, New York, NY 10013, and outside the United States by Springer-Verlag GmbH & Co. KG, Tiergartenstr. 17, 69112 Heidelberg, Germany.

In the United States: phone 1-800-SPRINGER, fax 201-348-4505, e-mail orders@springer-ny.com, or visit <http://www.springer-ny.com>. Outside the United States: fax +49 6221 345229, e-mail orders@springer.de, or visit <http://www.springer.de>.

For information on translations, please contact Apress directly at 2560 Ninth Street, Suite 219, Berkeley, CA 94710. Phone 510-549-5930, fax 510-549-5939, e-mail info@apress.com, or visit <http://www.apress.com>.

The information in this book is distributed on an “as is” basis, without warranty. Although every precaution has been taken in the preparation of this work, neither the author(s) nor Apress shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in this work.

The source code for this book is available to readers at <http://www.apress.com> in the Downloads section.

*In loving memory of “the Weens”—sweet little Shylow and funny little Chloe—
who graced our lives with unconditional love for almost 17 years.*

Contents at a Glance

About the Author.....	xxiii
About the Technical Reviewer.....	xxv
Acknowledgments.....	xxvii
Preface.....	xxix
Introduction.....	xxxi

PART 1 ■ ■ ■ The ABCs of Objects

CHAPTER 1	Abstraction and Modeling.....	3
CHAPTER 2	Some Java Basics.....	15
CHAPTER 3	Objects and Classes.....	65
CHAPTER 4	Object Interactions.....	95
CHAPTER 5	Relationships Between Objects.....	167
CHAPTER 6	Collections of Objects.....	213
CHAPTER 7	Some Final Object Concepts.....	273

PART 2 ■ ■ ■ Object Modeling 101

CHAPTER 8	The Object Modeling Process in a Nutshell.....	333
CHAPTER 9	Formalizing Requirements Through Use Cases.....	343
CHAPTER 10	Modeling the Static/Data Aspects of the System.....	355
CHAPTER 11	Modeling the Dynamic/Behavioral Aspects of the System.....	409
CHAPTER 12	Wrapping Up Our Modeling Efforts.....	431

PART 3 ■ ■ ■ Translating an Object Blueprint into Java Code

CHAPTER 13	Rounding Out Your Java Knowledge	441
CHAPTER 14	Transforming Your Model into Java Code	567
CHAPTER 15	Rounding Out Your Application, Part 1: Adding a Data Access Layer	619
CHAPTER 16	Rounding Out Your Application, Part 2: Adding a Presentation Layer.....	681
CHAPTER 17	SRS, Take 3: Adding a GUI	773
CHAPTER 18	Next Steps	831

PART 4 ■ ■ ■ Appendixes

APPENDIX A	Suggestions for Using This Book As a Textbook	839
APPENDIX B	Alternative Case Studies	843
APPENDIX C	Setting Up Your Java Development Environment	849
APPENDIX D	Downloading and Compiling the Book's Source Code	865
APPENDIX E	Note to Experienced C++ Programmers	867
APPENDIX F	How Polymorphism Works Behind the Scenes (Static vs. Dynamic Binding)	873
APPENDIX G	Collections Prior to J2SE 5.0	879
APPENDIX H	Programming 101, and the Role of a Compiler	891
INDEX		897

Contents

About the Author.....	xxiii
About the Technical Reviewer.....	xxv
Acknowledgments.....	xxvii
Preface.....	xxix
Introduction.....	xxxi

PART 1 ■ ■ ■ The ABCs of Objects

■ CHAPTER 1	Abstraction and Modeling	3
	Simplification Through Abstraction.....	3
	Generalization Through Abstraction.....	4
	Organizing Abstractions into Classification Hierarchies.....	5
	Abstraction As the Basis for Software Development.....	8
	Reuse of Abstractions.....	9
	Inherent Challenges.....	10
	What Does It Take to Be a Successful Object Modeler?.....	11
	Summary.....	12
■ CHAPTER 2	Some Java Basics	15
	Why Java?.....	15
	Java Is Architecture Neutral.....	16
	Java Provides “One-Stop Shopping”.....	20
	Java Is Object-Oriented from the Ground Up.....	22
	Practice Makes Perfect.....	23
	Java Is an Open Standard.....	23
	Java Is Free!.....	23
	A Reminder Regarding Pseudocode vs. Real Java Code.....	24
	Anatomy of a Simple Java Program.....	24
	Comments.....	24
	The Class Declaration.....	26
	The main Method.....	27

The “Mechanics” of Java	28
Compiling Java Source Code into Bytecode	30
Executing Bytecode	31
A Behind-the-Scenes Look at the JVM	32
Primitive Types	32
Variables	33
Variable Naming Conventions	34
Variable Initialization	35
The String Type	36
Case Sensitivity	37
Java Expressions	38
Arithmetic Operators	38
Relational and Logical Operators	40
Evaluating Expressions and Operator Precedence	41
The Type of an Expression	42
Automatic Type Conversions and Explicit Casting	42
Loops and Other Flow-Control Structures	44
if Statements	45
switch Statements	47
for Statements	49
while Statements	51
Jump Statements	52
Block-Structured Languages and the Scope of a Variable	53
Printing to the Screen	54
print vs. println	56
Escape Sequences	57
Elements of Java Style	58
Proper Use of Indentation	58
Use Comments Wisely	61
Placement of Braces	61
Descriptive Variable Names	62
Summary	63
 CHAPTER 3 Objects and Classes	 65
Software at Its Simplest	65
Functional Decomposition	66
The Object-Oriented Approach	67
What Is an Object?	68
State/Data/Attributes	69
Behavior/Operations/Methods	70

What Is a Class?	71
A Note Regarding Naming Conventions	73
Declaring a Class, Java Style	73
Instantiation	74
Encapsulation	75
User-Defined Types and Reference Variables	76
Naming Conventions for Reference Variables	77
Instantiating Objects: A Closer Look	77
Garbage Collection	84
Objects As Attributes	85
A Compilation Trick: “Stubbing Out” Classes	88
Composition	89
The Advantages of References As Attributes	91
Three Distinguishing Features of an Object-Oriented Programming Language	92
Summary	92

CHAPTER 4 Object Interactions

Events Drive Object Collaboration	95
Declaring Methods	97
Method Headers	98
Method Naming Conventions	98
Passing Arguments to Methods	99
Method Return Types	100
An Analogy	101
Method Bodies	102
Features May Be Declared in Any Order	103
return Statements	104
Methods Implement Business Rules	107
Objects As the Context for Method Invocation	108
Java Expressions, Revisited	111
Capturing the Value Returned by a Method	111
Method Signatures	112
Choosing Descriptive Method Names	114
Method Overloading	115
Message Passing Between Objects	116
Delegation	118
Obtaining Handles on Objects	119
Objects As Clients and Suppliers	122

Information Hiding/Accessibility	124
Public Accessibility	125
Private Accessibility	126
Publicizing Services	127
Method Headers, Revisited	128
Accessing the Features of a Class from Within Its Own Methods	129
Accessing Private Features from Client Code	132
Declaring Accessor Methods	133
Recommended “Get”/“Set” Method Headers	134
The “Persistence” of Attribute Values	137
Using Accessor Methods from Client Code	137
The Power of Encapsulation Plus Information Hiding	138
Preventing Unauthorized Access to Encapsulated Data	139
Helping to Ensure Data Integrity	139
Limiting “Ripple Effects” When Private Features Change	140
Using Accessor Methods from Within a Class’s Own Methods ...	143
Exceptions to the Public/Private Rule	147
Exception #1: Internal Housekeeping Attributes	147
Exception #2: Internal Housekeeping Methods	148
Exception #3: “Read-Only” Attributes	149
Exception #4: Public Attributes	150
Constructors	150
Default Constructors	150
Writing Our Own Explicit Constructors	151
Passing Arguments to Constructors	152
Replacing the Default Parameterless Constructor	153
More Elaborate Constructors	154
Overloading Constructors	155
An Important Caveat Regarding the Default Constructor	157
Using the “this” Keyword to Facilitate Constructor Reuse	158
Software at Its Simplest, Revisited	161
Summary	163

CHAPTER 5 Relationships Between Objects

Associations and Links	167
Multiplicity	170
Multiplicity and Links	171
Aggregation and Composition	173

Inheritance	174
Responding to Shifting Requirements with a New Abstraction	175
(Inappropriate) Approach #1: Modify the Student Class	176
(Inappropriate) Approach #2: “Clone” the Student Class to Create a GraduateStudent Class	179
The Proper Approach (#3): Taking Advantage of Inheritance	180
The “is a” Nature of Inheritance	181
The Benefits of Inheritance	183
Class Hierarchies	184
The Object Class	186
Is Inheritance Really a Relationship?	186
Avoiding “Ripple Effects” in a Class Hierarchy	187
Rules for Deriving Classes: The “Do’s”	187
Overriding	188
Reusing Superclass Behaviors: The “super” Keyword	191
Rules for Deriving Classes: The “Don’ts”	194
Private Features and Inheritance	196
Inheritance and Constructors	199
A Few Words About Multiple Inheritance	205
Three Distinguishing Features of an OOPL, Revisited	209
Summary	209

CHAPTER 6 Collections of Objects	213
What Are Collections?	213
Collections Are Defined by Classes and Must Be Instantiated	214
Collections Organize References to Other Objects	214
Collections Are Encapsulated	216
Three Generic Types of Collection	216
Ordered Lists	216
Dictionaries	217
Sets	218
Arrays As Simple Collections	219
Declaring and Instantiating Arrays	220
Accessing Individual Array Elements	221
Initializing Array Contents	222
Manipulating Arrays of Objects	223

A More Sophisticated Type of Collection: The ArrayList Class	227
Using the ArrayList Class: An Example	228
Import Directives and Packages	228
The Namespace of a Class	231
User-Defined Packages and the Default Package	233
Generics	234
ArrayList Features	235
Iterating Through ArrayLists	238
Copying the Contents of an ArrayList into an Array	238
The HashMap Collection Class	240
The TreeMap Class	246
The Same Object Can Be Simultaneously Referenced by Multiple Collections	248
Inventing Our Own Collection Types	249
Approach #1: Designing a New Collection Class from Scratch	249
Approach #2: Extending a Predefined Collection Class (MyIntCollection)	249
Approach #3: Encapsulating a Standard Collection (MyIntCollection2)	254
Trade-offs of Approach #2 vs. Approach #3	258
Collections As Method Return Types	259
Collections of Derived Types	260
Revisiting Our Student Class Design	261
The courseLoad Attribute of Student	262
The transcript Attribute of Student	262
The transcript Attribute, Take 2	265
Our Completed Student Data Structure	270
Summary	270

■ CHAPTER 7 **Some Final Object Concepts**

Polymorphism	274
Polymorphism Simplifies Code Maintenance	279
Three Distinguishing Features of an Object-Oriented Programming Language	281
The Benefits of User-Defined Types	281
The Benefits of Inheritance	281
The Benefits of Polymorphism	281

Abstract Classes	282
Implementing Abstract Methods	286
Abstract Classes and Instantiation	287
Declaring Reference Variables of Abstract Types	289
An Interesting Twist on Polymorphism	289
Interfaces	290
Implementing Interfaces	293
Another Form of the “Is A” Relationship	296
Interfaces and Casting	297
Implementing Multiple Interfaces	300
Interfaces and Casting, Revisited	303
Interfaces and Instantiation	303
Interfaces and Polymorphism	304
The Importance of Interfaces	304
Static Features	313
Static Variables	314
A Design Improvement: Burying Implementation Details	317
Static Methods	318
Restrictions on Static Methods	319
Revisiting the Syntax of Print Statements	321
Utility Classes	321
The final Keyword	322
The Static Import Facility	325
Custom Utility Classes	326
Summary	327

PART 2 ■ ■ ■ Object Modeling 101

■ CHAPTER 8 The Object Modeling Process in a Nutshell	333
The “Big Picture” Goal of Object Modeling	333
Modeling Methodology = Process + Notation + Tool	334
Jacquie’s Recommended Object Modeling Process, in a Nutshell	337
Thoughts Regarding Object Modeling Software Tools	338
A Reminder	340
Summary	341

CHAPTER 9	Formalizing Requirements Through Use Cases	343
	What Are Use Cases?	344
	Functional vs. Technical Requirements	344
	Involving the Users	345
	Actors	346
	Identifying Actors and Determining Their Roles	346
	Diagramming a System and Its Actors	347
	Specifying Use Cases	350
	Matching Up Use Cases with Actors	351
	To Diagram or Not to Diagram?	351
	Summary	353
CHAPTER 10	Modeling the Static/Data Aspects of the System	355
	Identifying Appropriate Classes	356
	Noun Phrase Analysis	356
	Refining the Candidate Class List	361
	Revisiting the Use Cases	365
	Producing a Data Dictionary	367
	Determining Associations Between Classes	368
	Identifying Attributes	372
	UML Notation: Modeling the Static Aspects of an Abstraction	372
	Classes, Attributes, and Operations	372
	Relationships Between Classes	376
	Reflecting Multiplicity	382
	Object Diagrams	386
	Associations As Attributes	388
	Information “Flows” Along an Association “Pipeline”	389
	“Mixing and Matching” Relationship Notations	395
	Association Classes	397
	Our “Completed” Student Registration System Class Diagram	400
	Metadata	406
	Summary	407
CHAPTER 11	Modeling the Dynamic/Behavioral Aspects of the System	409
	How Behavior Affects State	410
	Events	412

Scenarios	415
Scenario #1 for the “Register for a Course” Use Case	416
Scenario #2 for the “Register for a Course” Use Case	418
Sequence Diagrams	419
Using Sequence Diagrams to Determine Methods	424
Communication Diagrams	426
Revised SRS Class Diagram	428
Summary	429

CHAPTER 12 Wrapping Up Our Modeling Efforts	431
Testing the Model	431
Revisiting Requirements	432
Reusing Models: A Word About Design Patterns	434
Summary	437

PART 3 ■ ■ ■ Translating an Object Blueprint into Java Code

CHAPTER 13 Rounding Out Your Java Knowledge	441
Java-Specific Terminology	441
Java Application Architecture	444
Java Archive (JAR) Files	446
Creating a JAR File	447
Inspecting the Contents of a JAR File	448
Using the Bytecode Contained Within a JAR File	448
Extracting Content from a JAR File	449
“Jarring” Entire Directory Hierarchies	449
Javadoc Comments	450
The Object Nature of Strings	457
Operations on Strings	457
Strings Are Immutable	460
The StringBuffer Class	462
The StringTokenizer Class	462
Instantiating Strings and the String Literal Pool	465
Testing the Equality of Strings	469
Message Chains	470
Object Self-Referencing with “this”	472

Java Exception Handling	473
The Mechanics of Exception Handling	475
Catching Exceptions	484
Interpreting Exception Stack Traces	489
The Exception Class Hierarchy	490
Catching the Generic Exception Type	493
Compiler Enforcement of Exception Handling	494
Taking Advantage of the Exception That We've "Caught"	496
Nesting of try/catch Blocks	496
User-Defined Exception Types	497
Throwing Multiple Types of Exceptions	500
Enum(eration)s	501
Enumerating Choices Prior to J2SE 5.0	508
Providing Input to Command Line–Driven Programs	509
Accepting Command-Line Arguments: The args Array	510
Introducing Custom Command-Line Flags to Control a Program's Behavior	511
Using the Wrapper Classes for Input Conversion	516
Accepting Keyboard Input	518
Features of the Object Class	522
Determining the Class That an Object Belongs To	522
Testing the Equality of Objects	523
Overriding the equals Method	527
Overriding the toString Method	530
A Deeper Look Behind the Scenes of the JVM	532
The JVM's Class Loader	532
The -verbose Option	533
The Static Nature of main(...)	537
Import Directives, Revisited	537
Static Initializers	539
The Date Class	541
Accessibility, Revisited	547
Default Accessibility of a Feature	547
Public vs. Nonpublic Classes	547
Variable Initialization, Revisited	552
Inner Classes	554
Additional J2SE 5.0 Enhancements	557
Formatted Output	558
Formatted Input	559
Variable Arguments (Varargs)	560
Summary	564

CHAPTER 14 Transforming Your Model into Java Code	567
Suggestions for Getting the Maximum Value from This and Subsequent Chapters	567
The SRS Class Diagram Revisited	568
The Person Class (Specifying Abstract Classes)	571
The Student Class (Reuse Through Inheritance, Abstract Class Extension, Delegation)	574
The Professor Class (Bidirectional Relationships)	582
The Course Class (Reflexive Relationships, Unidirectional Relationships)	584
The Section Class (Representing Association Classes, Public Static Final Attributes, Enums)	587
Delegation Revisited	595
The ScheduleOfClasses Class	600
The TranscriptEntry Association Class (Static Methods)	602
The Transcript Class	605
The SRS Driver Program	606
The Importance of Model–View and Model–Data Layer Separation	617
Summary	618
 CHAPTER 15 Rounding Out Your Application, Part 1: Adding a Data Access Layer	619
An Overview of Upcoming SRS Enhancements	621
Approaches to Object Persistence	621
The Basics of File I/O in Java	624
Reading from a File	624
Writing to a File	626
Exception Handling with File I/O	629
Populating the Main SRS Collections	631
The ScheduleOfClasses Collection	631
Two New Encapsulated Collection Classes: CourseCatalog and Faculty	631
Initializing the SRS Collections from Record-Oriented Data Files	634
Persisting Student Data	637
Read vs. Write Access to the Data Files	639
Configuring Applications with the Java Properties Class	639
The FileNames.properties File	640
Accessing System Properties	641

Defining Custom Exceptions for the SRS.....	644
Encapsulating Persistence Details	645
Introducing the SRSDataAccess Class.....	645
Initializing the ScheduleOfClasses Collection	648
Initializing the Faculty Collection.....	652
Initializing the CourseCatalog Collection	655
Initializing a Student's State.....	657
Persisting the State of a Student.....	660
Streamlining the SRS Driver Class	661
The Importance of Model–Data Access Layer Separation	669
Objects and Databases.....	670
Using the JDBC API: A Conceptual Overview.....	671
Retrofitting the SRS with Database Access	674
Our SRS Modifications, Revisited	678
Summary	679

CHAPTER 16 Rounding Out Your Application, Part 2:	
Adding a Presentation Layer	681
Java GUIs: a Primer.....	682
Components	682
Containers.....	683
Model–View Separation	684
AWT vs. Swing Components	688
Crafting the View/Presentation of a Java GUI.....	692
JFrames.....	692
Positioning a Frame on the Screen.....	696
Explicitly Positioning a Frame on the Screen	697
Centering a Frame on the Screen	698
Adding Components to a JFrame	700
JPanels	702
Common Component Properties and Behaviors.....	702
Layout Fundamentals	706
BorderLayout	706
GridLayout.....	711
FlowLayout	717
JLabels	719
JTextFields and JPasswordField	719
JButtons	722
JLists	723

A Simple Calculator Example	726
An Improved Application Architecture for GUIs	729
Adding a “Test Scaffold” main() Method	733
Other Interesting AWT/Swing Components to Explore	734
Java Event Handling	735
Events: Basic Concepts	735
Basic Event Types	738
Creating and Registering Listeners	739
Adding Behavior to Our Calculator	742
Combining Listeners: Utilizing the ActionEvent	746
Closing a Window, Revisited	748
Adapter Classes vs. Listener Interfaces	751
Selecting an Item from a JList	754
More Container Types	761
JDialog	761
One-Step Dialog Boxes with JOptionPane	766
Reusable Custom Components	769
Summary	771
 ■ CHAPTER 17 SRS, Take 3: Adding a GUI	773
Our SRS Code Road Map	773
Preparing a Concept of Operations	775
The SRS Concept of Operations	776
The MainFrame Class	783
Planning the “Look” of MainFrame: Getting Creative with GridLayout	783
Coding the “Look” of MainFrame	784
Adding Behaviors to MainFrame	789
The PasswordPopup Class: Sharing Information Across Windows/Classes	803
The SRS Driver Class, Significantly Streamlined	805
An Overview of J2EE	806
The Downside of Desktop Application Deployment	807
The Upside of Web Deployment	808
What Is J2EE?	813
What Is a Servlet?	814
What Is a JavaServer Page?	816
J2EE, Model View Controller–Style	819
N-Tier Architectures	822

What Are EJBs?	822
Dispelling J2EE Myths	825
The Relevance of What You've Learned in This Book to J2EE	826
Review of the Architecture of an "Industrial-Strength" Java Application	826
Summary	829

CHAPTER 18 Next Steps	831
Jacque's "Tried and True" Approach to Learning Java Properly	832
Taming the Technology Tidal Wave	833
Other Recommended Reading	834
Your Comments, Please!	835

PART 4 ■ ■ ■ **Appendixes**

APPENDIX A Suggestions for Using This Book As a Textbook	839
Recommended Teaching Approaches	839
Suitability of Java As a Teaching Language	840
Some Final Recommendations	840
APPENDIX B Alternative Case Studies	843
Case Study #1: Prescription Tracking System	843
Background	843
Simplifying Assumptions	844
Case Study #2: Conference Room Reservation System	845
Background	845
Goals for the System	846
Case Study #3: Blue Skies Airline Reservation System	847
Background	847
Other Simplifying Assumptions	848
APPENDIX C Setting Up Your Java Development Environment	849
The Java Software Development Kit	849
Testing Your Installation	850

Troubleshooting Your Installation	852
PATH and CLASSPATH Settings	852
Common Compilation Errors	857
Common Run-Time Errors	860
Using the Online Java Documentation with Windows	861
Special Tips for Using Microsoft Windows Command Prompts	862
Capturing Program Output to a File	863
Resizing the Command Prompt Window	863
■ APPENDIX D Downloading and Compiling the Book's Source Code ..	865
■ APPENDIX E Note to Experienced C++ Programmers	867
No More Pointers!	867
Dynamic vs. Static Object Creation	868
Exception Handling	869
“Breaking the OO Rules” with C++	869
Platform Portability	870
Abstract Methods and Abstract Classes	870
Other Simplifications	870
■ APPENDIX F How Polymorphism Works Behind the Scenes (Static vs. Dynamic Binding)	873
Static Binding	874
Dynamic Binding	875
■ APPENDIX G Collections Prior to J2SE 5.0	879
Summary of 5.0 Collection Enhancements	879
Constraining Collection Contents	880
Iterating Through Collections	881
Managing Primitive Types with Collections	884
The Iterator Class	886
Using the -Xlint:unchecked Compiler Option	888
■ APPENDIX H Programming 101, and the Role of a Compiler	891
A LEGOs Analogy	891
The Compiler's Job in a Nutshell	892
■ INDEX	897

About the Author



JACQUIE BARKER is a professional software engineer, author, and adjunct faculty member at both George Mason University in Fairfax, Virginia, and The George Washington University in Washington, D.C. With over 28 years of experience as a hands-on software developer and project manager, Jacquie has spent the past 12 years focusing on object technology, and becoming proficient as an object modeler and Sun Microsystems Certified Java programmer. She is the founder of ObjectStart LLC, an object technology mentorship and training firm, and she is also a senior member of the technical staff of Technology Associates, Inc. in Herndon, Virginia.

Jacquie earned a bachelor of science degree in computer engineering with highest honors from Case Western Reserve University in Cleveland, Ohio, where she was also inducted into the Tau Beta Pi National Engineering Honor Society. She later received a master of science degree in computer science, focusing on software systems engineering, from the University of California, Los Angeles, and has subsequently pursued postgraduate studies in information technology at George Mason University.

Jacquie's winning formula for teaching object fundamentals continues to receive praise from readers around the world, and *Beginning Java Objects: From Concepts to Code* has been adopted by many universities as a key textbook in their core IT curricula as a result. Her latest book, *Taming the Technology Tidal Wave: Practical Career Advice for IT Professionals*, is a light-hearted guide to maintaining leading-edge technical skills, and is similarly getting rave reviews.

On a personal note, Jacquie's passions include her husband, Steve; pet rats and miniature longhaired dachshunds (aka wiener dogs); teaching; writing; and programming in Java. When not engaged in computer-related pursuits, Jacquie and Steve enjoy taking motorcycle road trips through the Virginia countryside, tandem bicycling, enjoying quiet vacations at the lake, and spending quality time with family and friends.

Please visit Jacquie's web sites, <http://objectstart.com> and <http://techtidalwave.com>, for more information on her various publications and service offerings.

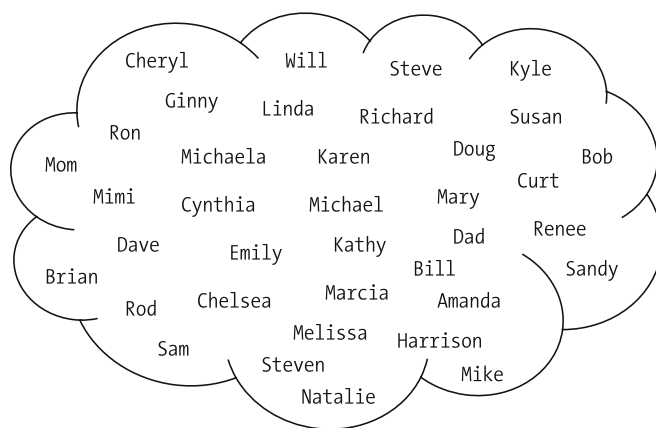
About the Technical Reviewer

■ **JAMES HUDDLESTON** is an independent consultant with over 30 years' experience in information technology. Revision author of Apress's *Beginning C# Databases: From Novice to Professional*, he's been a technical reviewer or editor for dozens of books on diverse computer topics. When not having fun writing programs for a living or translating Homer as a hobby, he delights in helping authors make good books better—and even more in playing with his three children, Jared, Quinn, and Tess.

Acknowledgments

My sincerest thanks go to

- James Huddleston, my technical reviewer, for his wise and thoughtful guidance in shaping this second edition of *Beginning Java Objects: From Concepts to Code*. James, you are a true Renaissance man and a delight to work with—not to mention **brilliant!**—and I look forward to many future collaborations with you.
- Beckie Stones, Nicole LeClerc, Katie Stence, and Steve Anglin, for their superb support in producing *Beginning Java Objects: From Concepts to Code, Second Edition*. I couldn't have done it without you! (Advance thanks to Paul Carlstroem and Stephanie Parker, as well, for lending their marketing wisdom in making my book a spectacular success.)
- Gary Cornell, author and publisher of Apress, for continuing to be a fan of and advocate for *Beginning Java Objects*.
- My friends Cathy McCabe and Brian and Cynthia Coleman, for informally reviewing my book and providing me with invaluable insights into how my message could be refined.
- Dr. Claudio Cioffi and my spring semester 2005 Computational Social Sciences 605 students at George Mason University, for helping me to hone my message for a broader audience.
- The many loved ones who patiently awaited my return from the “social abyss” known as writing a book:



And most important, to the love of my life and my best friend, my husband Steve, for being such a supportive and understanding partner. I love sharing my life with you, “Klemmie”!

Preface

Welcome to the second edition of *Beginning Java Objects: From Concepts to Code!* Since the first edition of *Beginning Java Objects* was published back in November 2000, I've been delighted by the many emails and positive reviews that I've received from readers who found my book to be a perfect "jump-start" into Java and object-oriented programming.

My book is based on timeless principles that are language version independent, which means that it needn't be revised every time a new version of Java is released by Sun Microsystems. That being said, Java 2 Platform, Standard Edition (J2SE) 5.0 introduced some important new features that have significantly increased the power and versatility of Java as an object-oriented programming language. For this reason, I've deemed this to be the right time to "freshen" *Beginning Java Objects* in the form of a second edition.

I've also continued to teach the material of *Beginning Java Objects* since the first edition was published, and as a result of working with many students in both university and corporate settings, I've refined my message in important ways. Teaching, as learning, is a work in progress; one's message can never be 100% complete!

Noteworthy enhancements to *Beginning Java Objects* in this second edition include

- Emphasis on getting hands-on experience with Java much sooner (as of Chapter 2 versus Chapter 13 of the previous edition)
- A significantly improved discussion of model–data layer separation in Chapter 15, including a conceptual introduction to the JDBC API (used to communicate with ODBC-compliant databases)
- Greater focus on model–view separation, including a conceptual introduction to J2EE technology in Chapter 17
- Emphasis on the significant enhancements made to the Java language as of J2SE 5.0, highlighting their significance in OO terms

Recognizing that *Beginning Java Objects* has enjoyed widespread academic adoption as a textbook, I've made certain that the chapter outline of the second edition parallels that of the first edition, with only one exception: since the chapter formerly referred to as Chapter 16 had become so huge with this edition, I've split it into two chapters.

As always, I welcome reader feedback, and I hope to hear from you soon via either of my two websites, <http://objectstart.com> and <http://techtidalwave.com>.

Best regards,

Jacquie

Introduction

This is a book, first and foremost, about software objects: what they are, why they are so “magical” and yet so straightforward, and how you go about structuring a software application to use objects appropriately.

This is also a book about Java. It’s not a hard-core, “everything there is to know about Java” book, but rather a gentle yet comprehensive introduction to the language, with special emphasis on how to transition from an object model to a fully functional Java application—something that few, if any, other books provide.

Goals for This Book

My goals in writing this book (and, hopefully, yours for buying it) are to

- ***Make you comfortable with fundamental object-oriented (OO) terminology and concepts***
- ***Give you hands-on, practical experience with object modeling***—that is, with developing a “blueprint” that you can use as the basis for subsequently building an object-oriented software system
- ***Illustrate the basics of how such an object model is translated into a working software application—a Java application, to be specific***, although the techniques that you’ll learn for object modeling apply equally well to any OO language
- ***Help you to become proficient as a Java programmer along the way***

If you’re already experienced with the Java language (but not with object fundamentals), it’s critical to your successful use of the language that you learn about its object-oriented roots. On the other hand, if you’re a newcomer to Java, then this book will get you properly jump-started. ***Either way, this book is a must-read for anyone who wishes to become proficient with an OO programming language like Java.***

Just as important, this book is *not* meant to

- **Turn you into an overnight pro in object modeling.** Like all advanced skills, becoming totally comfortable with object modeling takes two things: a good theoretical foundation and a lot of practice. I give you the foundation in this book, including an introduction to the Unified Modeling Language (UML), the industry standard for rendering an object-oriented “blueprint” of a software application. But the only way you’ll really get to be proficient with object modeling is by participating in OO modeling and development projects over time.

My book will give you the skills and, hopefully, the *confidence* to begin to apply object techniques in a professional setting, which is where your real learning will take place, particularly if you have an OO-experienced mentor to guide you through your first “industrial-strength” project.

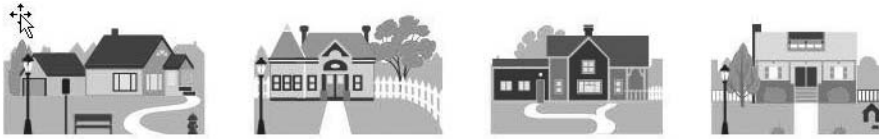
- **Teach you everything you’ll ever need to know about Java.** Java is a very rich language, consisting of hundreds of core classes and literally thousands of operations that can be performed with and by these classes. If Java provides a dozen alternative ways to do something in particular, I explain the one or two ways that I feel best suit the problem at hand, to give you an appreciation for how things are done. Nonetheless, you’ll definitely see enough of the Java language in this book to prepare you for a role as a professional Java programmer.

Armed with the knowledge you’ll gain from this book, you’ll be poised and ready to appreciate a more thorough treatment of Java such as that offered by one of the many other Java references that are presently on the market, or a deeper review of object modeling techniques from an in-depth UML reference.

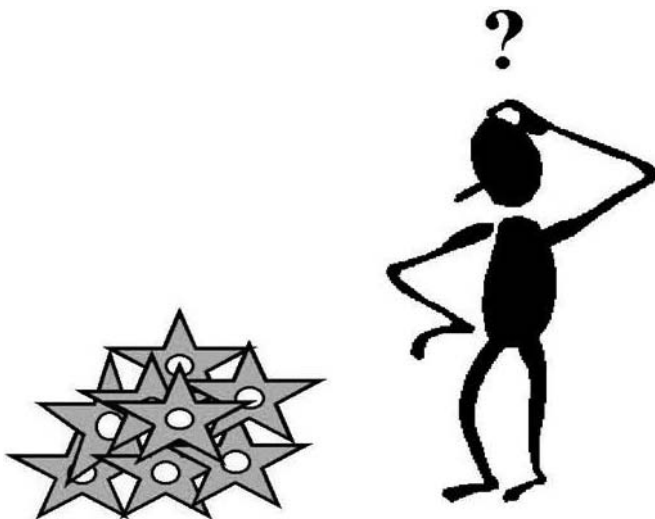
Why Is Understanding Objects So Critical to Being a Successful OO Programmer?

Time and again, I meet software developers—at my place of employment, at clients’ offices, at professional conferences, and on college campuses—who have attempted to master an object-oriented programming language like Java by taking a course in Java, reading a book about Java, or installing and using a Java integrated development environment (IDE) such as JBuilder or Sun One Studio. However, there is something fundamentally missing from virtually all of these approaches: a basic understanding of what objects are all about and, more important, *knowledge of how to structure a software application from the ground up to make the most of objects.*

Imagine that you’ve been asked to build a house, and that you know the basics of home construction. In fact, you’re a world-renowned homebuilder whose services are in high demand. You’ve built homes of every possible architectural style, using every known type of building material: brick, lumber, stone, metal, etc. So, when your client tells you that he wants you to use a brand-new type of construction material that he’ll provide, you’re happy to oblige.



On the day construction is to begin, a truck pulls up at the building site and unloads a large pile of odd-looking, star-shaped blue blocks with holes in the middle. You're totally baffled! You've built countless homes using more-familiar materials, but you don't have a *clue* about how to assemble a house using blue stars.



Scratching your head, you pull out a hammer and some nails and try to nail the blue stars together as if you were working with lumber, but the stars don't fit together very well. You then try to fill in the gaps with the same mortar that you would use to adhere bricks to one another, but the mortar doesn't stick to these blue stars very well. Because you're working under tight cost and schedule constraints for building this home for your client, however (and because you're too embarrassed to admit that you, as an *expert* homebuilder, don't know how to work with these modern materials), you press on. Eventually, you wind up with something that looks (on the surface, at least) like a house.



Your client comes to inspect the work, and he is terribly disappointed. One of the reasons he had selected blue stars as a construction material was that they are extremely energy efficient but, because you used nails and mortar to assemble the stars, they have lost a great deal of their inherent ability to insulate the home.

To compensate, your client asks you to replace all of the windows in the home with triple-pane thermal glass windows so that they will allow less heat to escape. *You're panicking at this point*—swapping out the windows will require you to literally rip the walls apart, destroying the house.

When you tell your customer this, he goes *ballistic*! Another reason he selected blue stars as a construction material was because of their modularity, and hence ease of accommodating design changes but, because of the ineffective way you assembled these stars, they've lost this flexibility as well.



This is, sad to say, the way many programmers wind up building an OO application when they don't have appropriate training in the fundamental properties of the building blocks of such an application—namely, software objects. Worse yet, the vast majority of would-be OO programmers are blissfully ignorant of the need to understand objects in order to program in an OO language. So, they take off programming with a language like Java and wind up with a far from ideal result: an application that lacks flexibility when an inevitable “midcourse correction” occurs in response to changing requirements after the application has been deployed.

Who Is This Book Written For?

Anyone who wants to get the most out of an object-oriented programming language like Java! It is for

- Anyone who has yet to tackle Java, but wants to get off on the right foot with the language
- Anyone who has ever purchased a book on Java—and who has read it faithfully—who understands the “bits and bytes” of the language, but doesn't quite know how to structure an application to best take advantage of Java's object-oriented features
- Anyone who has used a Java IDE but who really only knows how to drag and drop graphical user interface (GUI) components and to add a little bit of behind-the-scenes logic to buttons, menus, etc., without any real sense of how to properly structure the core of the application around objects
- Anyone who has built a Java application, but was disappointed with how difficult it was to maintain or modify it when new requirements were presented later in the application's life cycle
- Anyone who has previously learned something about object modeling, but is “fuzzy” on how to transition from an object model to real, live code (Java or otherwise)

The bottom line is that *anyone who really wants to master an OO language like Java must become an expert in objects first!*

In order to gain the most value from this book, you should have some programming experience under your belt; virtually any language will do. You should understand simple programming concepts such as

- Simple data types (integer, floating-point, etc.)
- Variables and their scope (including the notion of global data)
- Flow control (if ... then ... else statements, for/do/while loops, etc.)
- What arrays are, and how to use them
- The notion of a software function/subroutine/procedure: how to pass data in and get results back out

but you needn't have any prior exposure to Java. And, you needn't have ever been exposed to objects, either—in the software sense, at least! As you'll learn in Chapter 1, human beings naturally view the entire world from the perspective of objects.

Even if you've already developed a full-fledged Java application, it's certainly not too late to read this book if you still feel fuzzy when it comes to the object aspects of structuring an application; it ultimately makes someone a better Java programmer to know the “whys” of object orientation rather than merely the mechanics of the language. You'll most likely see some familiar landmarks (in the form of Java code examples) in this book, but you'll hopefully gain many new insights as you learn the rationale for why we do many of the things that we do when programming in Java (or any other OO programming language, for that matter).

Because this book has its roots in courses that I teach at the university level, it's ideally suited for use as a textbook for a semester-long university or advanced-placement high school course in either object modeling or Java programming. I've included some suggestions for how to use the book in that fashion in Appendix A.

What If You're Interested in Object Modeling, but Not Necessarily in Java Programming?

Will my book still be of value to you? **Definitely!** Even if you don't plan on making a career of programming (as is true of many of my object modeling students), I've found that being exposed to code examples written in an OO language like Java really helps to cement object concepts. So, you're encouraged to read Part 3, which steps through building a complete Java application, even if you never intend to set your hands to the keyboard for purposes of Java programming afterward.

How This Book Is Organized

The book is structured around three major topics, as follows.

Part 1: The ABCs of Objects

Before I dive into the how-to's of object modeling and the details of OO programming in Java, it's important that we all speak the same language with respect to objects. Part 1, consisting of Chapters 1 through 7, starts out slowly by defining basic concepts that underlie all software development approaches, OO or otherwise. But the chapters quickly ramp up to a discussion of advanced object concepts so that, by the end of Part 1, you should be “object-savvy.”

Part 2: Object Modeling 101

In Part 2—Chapters 8 through 12 of the book—I focus on the underlying principles of how and, more important, *why* we do the things that we do when we develop an object model of an application—principles that are common to all object modeling techniques. It's important to be conversant in the UML, and so I teach you the basics of the UML and use it for all of the concrete modeling examples in my book. Using the modeling techniques presented in these chapters, we'll develop an object model “blueprint” for a Student Registration System (SRS), the requirements specification for which is presented at the end of this introduction.

Part 3: Translating an Object Blueprint into Java Code

In Part 3 of the book—in Chapters 13 through 17—I illustrate how to render the SRS object model that we develop in Part 2 into a fully functioning Java application, complete with a GUI and a way to persist data from one user login to the next. All of the code examples presented in this section are available for download from the Apress web site (<http://www.apress.com>), and I strongly encourage you to download and experiment with this code.

The requirements specification for the SRS is written in the narrative style with which software system requirements are often expressed. You may feel confident that you could build an application today to solve this problem, but by the end of my book you should feel much more confident in your ability to build it as an **object-oriented** application. Three additional case studies—for a Prescription Tracking System, a Conference Room Reservation System, and an Airline Ticketing System, respectively—are presented in Appendix B; these serve as the basis for many of the exercises presented at the end of each chapter.

To round out the book, I've included a final chapter (Chapter 18) entitled “Next Steps,” which provides suggestions for how you might wish to continue your object-oriented discovery process after finishing this book. In that chapter, I provide a list of recommended books that will take you to the next level of proficiency, depending on what your intention is for applying what you've learned in my book.

Conventions

To help you get the most from the text and keep track of what's happening, I've used a number of conventions throughout the book, for instance:

Notes are shown in this fashion, and reflect important background information.

Here are styles you'll find in the text:

- When I introduce important words, I **bold** them to highlight them.
- I show filenames, URLs, and code within the text like so: `objectstart.com`
- I **bold** lines of code within long code passages if I want to call your attention to those lines in particular, for example:

```
// Bolding is used to call attention to new or significant code:
Student s = new Student();
// whereas unbolded code is code that's less important in the
// present context, or has been seen before.
int x = 3;
```

- I use *italic* versus regular code font to represent pseudocode:

```
// This is real code:
for (int i = 0; i <= 10; i++) {
    // This is pseudocode!
    compute the grade for the ith Student
}
```

Which Version of Java Is This Book Based On?

As with any programming language, from time to time new versions of Java will be released by Sun Microsystems. The **good news** is that, because I focus only on core Java language syntax in my book—language features that have been stable since Java's inception—this book isn't version specific. The techniques and concepts that you'll learn by reading my book will serve you equally well when new versions of Java appear.

That being said, Sun Microsystems introduced some important new features with J2SE 5.0 (alternatively known internally to Sun as Java 1.5.0) that significantly increased the power and versatility of Java as an object-oriented programming language. I've adopted the J2SE 5.0 approach to coding in this second edition of *Beginning Java Objects*, which impacts the code examples of Chapter 6 and Chapters 13 through 16. For readers who are still working with an earlier version of the Java language, I've included notes in these chapters to guide you in adapting the code for earlier versions of Java, if desired.

SORTING OUT JAVA NOMENCLATURE

Since Java first made its public debut as a programming language in 1995, there have been some discontinuities in the way in which Java language versions have been referred to.

The first version of the Java language was referred to as 1.0; this was followed (logically) by version 1.1. When version 1.2 arrived on the scene, however, significant differences had been made to the language in terms of how GUI events were handled, such that Sun Microsystems anointed this version “Java 2” (but still alternatively referred to it as 1.2.x). Versions 1.3.x and 1.4.x of Java were subsequently referred to as “Java 2 Platform (Version) 1.x.x”; however, when Java 1.5.0 was released, it again incorporated significant enough differences to warrant a new name: J2SE 5.0. (I discuss some of the more significant J2SE 5.0 enhancements in Chapters 6 and 13, and again in Appendix G.)

Table 1 summarizes the naming history of the Java language through Java 2 Platform 1.5.0.

Table 1. *Java Naming History**

Actual Version Number	Preferred Name
1.0	Java Language 1.0
1.1.x	Java Language 1.1
1.2.x	Java 2 Platform
1.3.x	Java 2 Platform 1.3.x
1.4.x	Java 2 Platform 1.4.x
1.5.0 (now 5.0.x)	Java 2 Platform 1.5.x (now Java 2 Platform 5.0.x)

** Version numbers in this table apply only to J2SE and J2EE.*

There have been similar discontinuities in the way in which the **Java Development Kit (JDK)** has been referred to. The JDK is the software “bundle” used by developers to build Java applications and consisting of

- The Java Virtual Machine (JVM)
- The Java compiler (javac)
- The Java Archive (jar) utility
- The Java documentation (javadoc) utility

etc. These discontinuities in naming are, in part, due to the fact that as of version 1.2 of the Java language, three Java “platforms” emerged:

- **Java 2 Platform, Micro Edition (J2ME):** A pared-down version of Java ideally suited to programming consumer electronic devices such as mobile phones, pagers, personal digital assistants (PDAs), etc.
- **Java 2 Platform, Standard Edition (J2SE):** The fundamental (and original) Java language and run time, used in building desktop applications such as we’ll be building for the SRS in Part 3 of this book. J2SE also serves as the foundation for the **Enterprise Edition** of Java (J2EE).
- **Java 2 Platform, Enterprise Edition (J2EE):** A set of component technologies used with J2SE to build multitier enterprise-level applications, as we discuss in Chapter 17.

Thus, the software bundle needed by Java developers has been alternatively referred to as follows:

- Java Development Kit (JDK)
- (Java) Software Development Kit (SDK)
- Java 2 Software Development Kit (Java 2 SDK)
- Java 2 Platform, Standard Edition (J2SE) Development Kit

etc. I'll refer to this software bundle as either the JDK or the SDK throughout the book.

Finally, there's the notion of the **Java Runtime Environment (JRE)**, that *subset* of the JDK consisting of the JVM, run-time class libraries, and Java application launcher that are needed to *execute* Java bytecode, but *omitting* developer tools such as the Java compiler (`javac`). For every release numbered 1.x.x of the JDK, there is a parallel release of the JRE.

A Final Thought Before You Get Started

A lot of the material in this book—particularly at the beginning of Part 1—may seem overly simplistic to experienced programmers. This is because much of object technology is founded on basic software engineering principles that have been in practice for many years and, in many cases, just repackaged slightly differently. There are indeed a few new tricks that make OO languages extremely powerful and that were virtually impossible to achieve with non-OO languages—**inheritance** and **polymorphism**, for example, which you'll learn more about in Chapters 5 and 7, respectively. (Such techniques can be simulated by hand in a non-OO language, just as programmers could program their own database management system (DBMS) from scratch instead of using a commercial product like Oracle, Sybase, or Microsoft SQL Server—but who'd *want* to?)

The biggest challenge for *experienced* programmers in becoming proficient with objects is reorienting the manner in which they think about the problem they will be automating.

- Software engineers/programmers who have developed applications using non-object-oriented methods often have to “unlearn” certain approaches used in the traditional methods of software analysis and design.
- Paradoxically, people just starting out as programmers (or as OO modelers) sometimes have an easier time when learning the OO approach to software development as their *only* approach.

Fortunately, the way we need to think about objects when developing software turns out to be the natural way that people think about the world in general. So, learning to “think” objects—and to program them in Java—is as easy as (Part) 1, (Part) 2, (Part) 3!

Please Tell Me What You Think

I, along with the Apress editorial and production staff, have worked hard to make this book as useful to you as possible, and we'd love to know what you think of it! We appreciate reader feedback, and take both constructive criticism and praise to heart in our future publishing efforts.

If you'd like to get in touch with anyone at Apress, please do so at info@apress.com or <http://www.apress.com>. You can contact me at jacquie@objectstart.com or via my web sites, <http://objectstart.com> and <http://techtidalwave.com>.

STUDENT REGISTRATION SYSTEM (SRS) CASE STUDY: SRS REQUIREMENTS SPECIFICATION

We have been asked to develop an automated Student Registration System (SRS). This system will enable students to register online for courses each semester, as well as track a student's progress toward completion of his or her degree.

When a student first enrolls at the university, the student uses the SRS to set forth a plan of study as to which courses he or she plans on taking to satisfy a particular degree program, and chooses a faculty advisor. The SRS will verify whether or not the proposed plan of study satisfies the requirements of the degree that the student is seeking. Once a plan of study has been established, then, during the registration period preceding each semester, the student is able to view the schedule of classes online, and choose whichever classes he or she wishes to attend, indicating the preferred section (day of week and time of day) if the class is offered by more than one professor. The SRS will verify whether or not the student has satisfied the necessary prerequisites for each requested course by referring to the student's online transcript of courses completed and grades received (the student may review his or her transcript online at any time).

Assuming that (a) the prerequisites for the requested course(s) are satisfied, (b) the course(s) meets one of the student's plan of study requirements, and (c) there is room available in each of the class(es), the student is enrolled in the class(es).

If (a) and (b) are satisfied, but (c) is not, the student is placed on a first-come, first-served waiting list. If a class/section that the student was previously waitlisted for becomes available (either because some other student has dropped the class or because the seating capacity for the class has been increased), the student is automatically enrolled in the waitlisted class, and an email message to that effect is sent to the student. It is the student's responsibility to drop the class if it is no longer desired; otherwise, he or she will be billed for the course.

Students may drop a class up to the end of the first week of the semester in which the class is being taught.

PART 1



The ABCs of Objects

Abstraction and Modeling

As human beings, we're flooded with information every day of our lives. Even if we could temporarily turn off all of the sources of “e-information” that are constantly bombarding us—emails, voicemails, news broadcasts, and the like—our five senses alone collect *millions* of bits of information per day just from our surroundings. Yet, we manage to make sense out of all of this information, typically without getting overwhelmed. Our brains naturally simplify the details of all that we observe so that these details are manageable through a process known as **abstraction**.

In this chapter, you'll learn

- How abstraction serves to simplify our view of the world
- How we organize our knowledge hierarchically to minimize the amount of information that we have to mentally juggle at any given time
- The relevance of abstraction to software development
- The inherent challenges that we face as software developers when attempting to model a real-world situation in software

Simplification Through Abstraction

Take a moment to look around the room in which you're reading this book. At first, you may think that there really aren't that many things to observe: some furniture, light fixtures, perhaps some plants, artwork, even some other people or pets. Maybe there is a window to gaze out of that opens up the outside world to observation.

Now look again. For each thing that you see, there are myriad details to observe: its size, its color, its intended purpose, the components from which it's assembled (the legs on a table, the lightbulbs in a lamp), etc. In addition, each one of these components in turn has details associated with it: the type of material used to make the legs of the table (wood or metal), the wattage of the lightbulbs, etc. Now factor in your other senses: the sound of someone snoring (hopefully not while reading this book!), the smell of popcorn coming from the microwave oven down the hall, and so forth. Finally, think about all of the unseen details of these objects: who manufactured them, or what their chemical, molecular, or genetic composition is.

It's clear that the amount of information to be processed by our brains is truly overwhelming! For the vast majority of people, this doesn't pose a problem, however, because we're innately skilled at **abstraction**, a process that involves recognizing and focusing on the important characteristics of a situation or object, and filtering out or ignoring all of the unessential details.

One familiar example of an abstraction is a road map. As an abstraction, a road map represents those features of a given geographic area relevant to someone trying to navigate with the map, perhaps by car: major roads and places of interest, obstacles such as large bodies of water, etc. Of necessity, a road map can't include every building, tree, street sign, billboard, traffic light, fast-food restaurant, etc. that physically exists in the real world. If it did, then it would be so cluttered as to be virtually unusable; none of the important features would stand out. Compare a road map with a topographical map, a climatological map, and a population density map of the same region: each abstracts out different features of the real world—namely, those relevant to the intended user of the map in question.

As another example, consider a landscape. An artist may look at the landscape from the perspective of colors, textures, and shapes as a prospective subject for a painting. A homebuilder may look at the same landscape from the perspective of where the best building site may be on the property, assessing how many trees will need to be cleared to make way for a construction project. An ecologist may closely study the individual species of trees and other plant/animal life for their biodiversity, with an eye toward preserving and protecting them, whereas a child may simply be looking at all of the trees in search of the best site for a tree house! Some elements are common to all four observers' abstractions of the landscape—the types, sizes, and locations of trees, for example—while others aren't relevant to all of the abstractions.

Generalization Through Abstraction

If we eliminate enough detail from an abstraction, it becomes generic enough to apply to a wide range of specific situations or instances. Such generic abstractions can often be quite useful. For example, a diagram of a generic cell in the human body, such as the one in Figure 1-1, might include only a few features of the structures that are found in an actual cell.

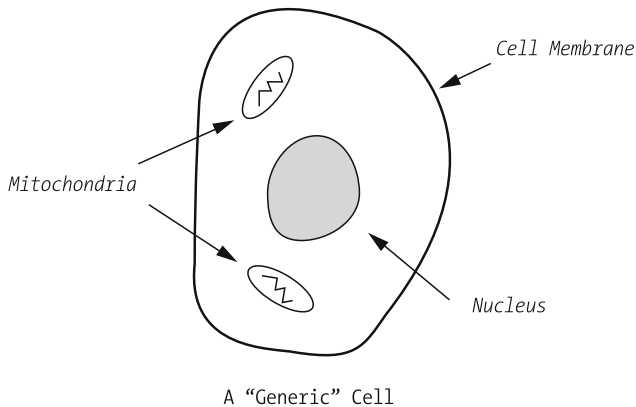


Figure 1-1. *A generic abstraction of a cell*

This overly simplified diagram doesn't look like a real nerve cell, or a real muscle cell, or a real blood cell; and yet, it can still be used in an educational setting to describe certain aspects of the structure and function of all of these cell types—namely, those features that the various cell types have in common.

The simpler an abstraction—that is, the fewer features it presents—the more general it is, and the more versatile it is in describing a variety of real-world situations. The more complex an abstraction, the more restrictive it is, and thus the fewer situations it is useful in describing.

Organizing Abstractions into Classification Hierarchies

Even though our brains are adept at abstracting concepts such as road maps and landscapes, that still leaves us with hundreds of thousands, if not millions, of separate abstractions to deal with over our lifetimes. To cope with this aspect of complexity, human beings systematically arrange information into categories according to established criteria; this process is known as **classification**.

For example, science categorizes all natural objects as belonging to either the animal, plant, or mineral kingdom. In order for a natural object to be classified as an animal, it must satisfy the following rules:

- It must be a living being.
- It must be capable of spontaneous movement.
- It must be capable of rapid motor response to stimulation.

The rules for what constitutes a plant, on the other hand, are different:

- It must be a living being (same as for an animal).
- It must lack an obvious nervous system.
- It must possess cellulose cell walls.

Given clear-cut rules such as these, placing an object into the appropriate category, or **class**, is rather straightforward. We can then “drill down,” specifying additional rules that differentiate various types of animal, for example, until we’ve built up a hierarchy of increasingly more complex abstractions from top to bottom. A simple example of an **abstraction hierarchy** is shown in Figure 1-2.

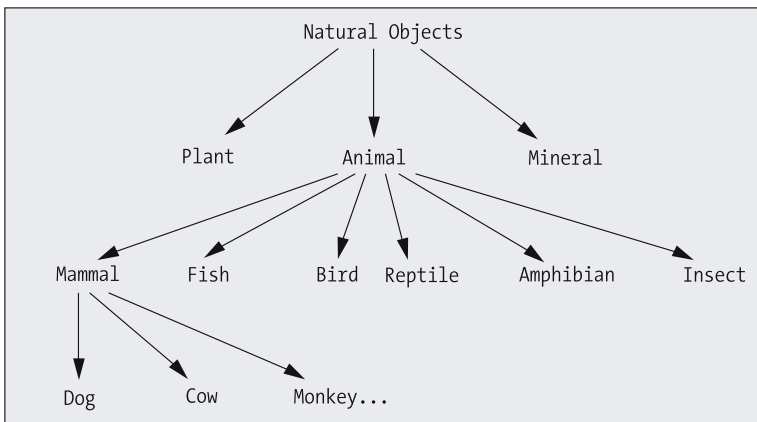


Figure 1-2. A simple abstraction hierarchy of natural objects

When thinking about an abstraction hierarchy such as the one shown in Figure 1-2, we mentally step up and down the hierarchy, automatically zeroing in on only the single layer or subset of the hierarchy (known as a **subtree**) that is important to us at a given point in time. For example, we may only be concerned with mammals and so can focus on the mammalian subtree, shown in Figure 1-3, temporarily ignoring the rest of the hierarchy.

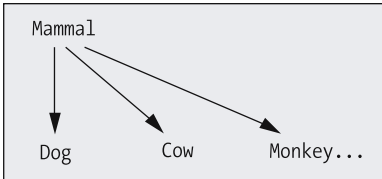


Figure 1-3. *Focusing on a small subset of the hierarchy is less overwhelming.*

By doing so, we automatically reduce the number of concepts that we mentally need to juggle at any one time to a manageable subset of the overall abstraction hierarchy; in our simplistic example, we're now dealing with only 4 concepts rather than the original 13. No matter how complex an abstraction hierarchy grows to be, it needn't overwhelm us if it's properly organized.

Coming up with precisely which rules are necessary to properly classify an object within an abstraction hierarchy isn't always easy. Take, for example, the rules we might define for what constitutes a bird: namely, something that

- Has feathers
- Has wings
- Lays eggs
- Is capable of flying

Given these rules, neither an ostrich nor a penguin could be classified as a bird, because neither can fly (see Figure 1-4).

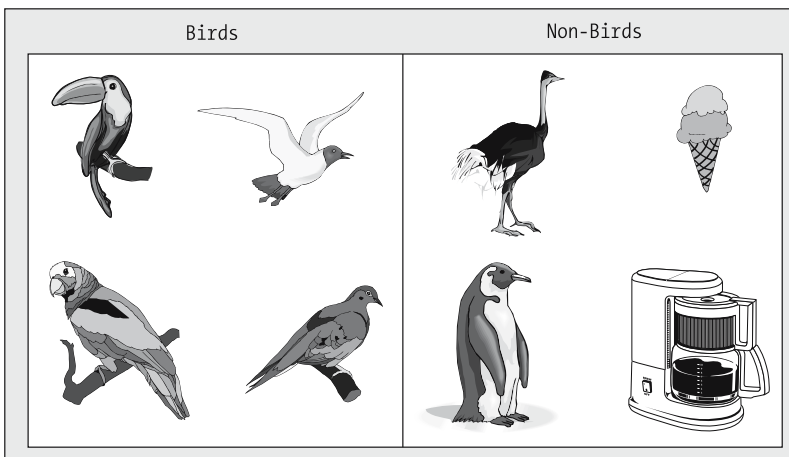


Figure 1-4. *Deriving the correct classification rules can be difficult.*
www.traintelco.com

If we attempt to make the rule set less restrictive by eliminating the “flight” rule, we’re left with

- Has feathers
- Has wings
- Lays eggs

According to this rule set, we now may properly classify both the ostrich and the penguin as birds, as shown in Figure 1-5.

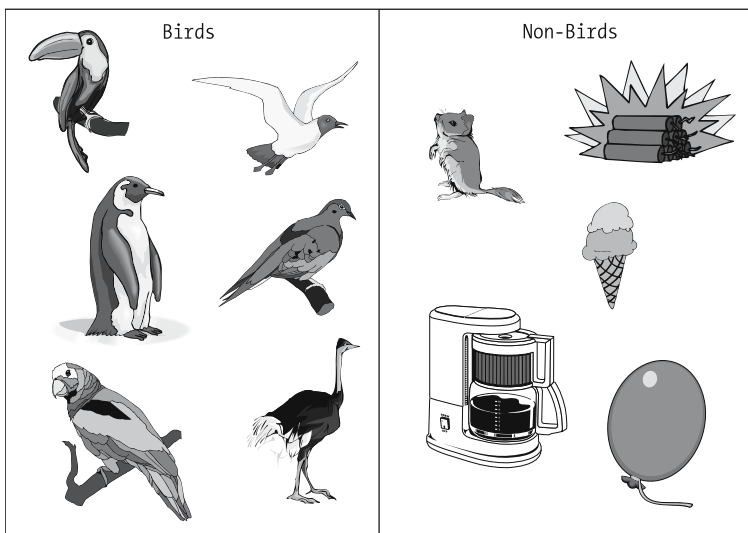


Figure 1-5. *Proper classification rules have been established.*

This rule set is still unnecessarily complicated, because as it turns out, the “lays eggs” rule is redundant: whether we keep it or eliminate it, it doesn’t change our decision of what constitutes a bird versus a non-bird. Therefore, we simplify the rule set once again:

- Has feathers
- Has wings

Feeling particularly daring (!), we try to take our simplification process one step further by eliminating yet another rule, defining a bird as something that

- Has wings

As Figure 1-6 shows, we’ve gone too far this time: the abstraction of a bird is now so general that we’d include airplanes, insects, and all sorts of other non-birds in the mix!

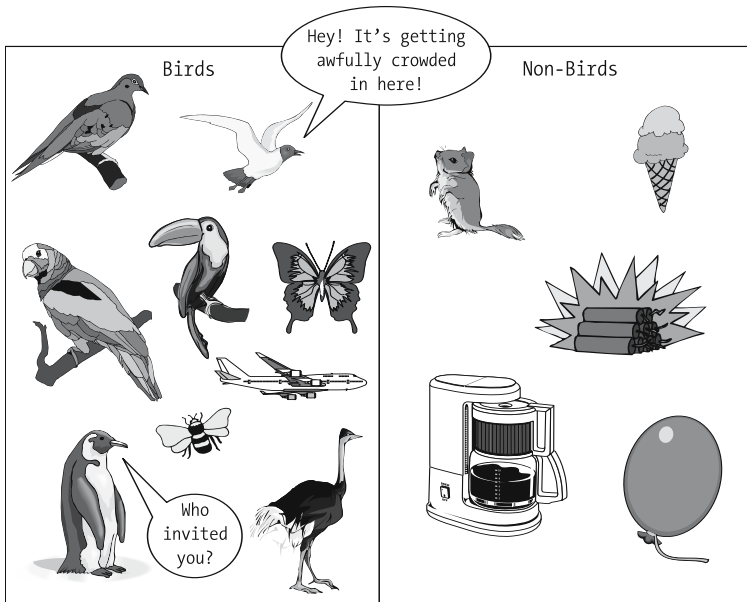


Figure 1-6. A rule set that is too relaxed is as much of a problem as an overly restrictive rule set.

The process of rule definition for purposes of categorization involves “dialing in” just the right set of rules—not too general, not too restrictive, and containing no redundancies—to define the correct membership in a particular class.

Abstraction As the Basis for Software Development

When pinning down the requirements for an information systems development project, we typically start by gathering details about the real-world situation on which the system is to be based. These details are usually a combination of

- Those that are explicitly offered to us as we interview the intended users of the system
- Those that we otherwise observe

We must make a judgment call as to which of these details are relevant to the system’s ultimate purpose. This is essential, as we can’t automate them all! To include too much detail is to overly complicate the resultant system, making it that much more difficult to design, program, test, debug, document, maintain, and extend in the future.

As with all abstractions, all of our decisions of inclusion versus elimination when building a software system must be made within the context of the overall purpose and **domain**, or subject matter focus, of the future system. When representing a person in a software system, for example, is their eye color important? How about their genetic profile? Salary? Hobbies? The answer is, *any* of these features of a person may be relevant or irrelevant, depending on whether the system to be developed is a

- Payroll system
- Marketing demographics system
- Optometrist's patient database
- FBI's "most wanted" tracking system

Once we've determined the essential aspects of a situation—something that we'll explore in Part 2 of this book—we can prepare a **model** of that situation. **Modeling** is the process by which we develop a pattern for something to be made. A blueprint for a custom home, a schematic diagram of a printed circuit, and a cookie cutter are all examples of such patterns. As we'll cover in Parts 2 and 3, an **object model** of a software system is such a pattern. Modeling and abstraction go hand in hand, because a model is essentially a physical or graphical portrayal of an abstraction; before we can model something effectively, we must have determined the essential details of the subject to be modeled.

Reuse of Abstractions

When learning about something new, we automatically search our mental archive for other abstractions/models that we've previously built and mastered, to look for similarities that we can build upon. When learning to ride a two-wheeled bicycle for the first time, for example, you may have drawn upon lessons that you learned about riding a tricycle as a child (see Figure 1-7). Both have handlebars that are used to steer; both have pedals that are used to propel the bike forward. Although the abstractions didn't match perfectly—a two-wheeled bicycle introduced the new challenge of having to balance yourself—there was enough of a similarity to allow you to draw upon the steering and pedaling expertise you already had mastered, and to focus on learning the new skill of how to balance on two wheels.

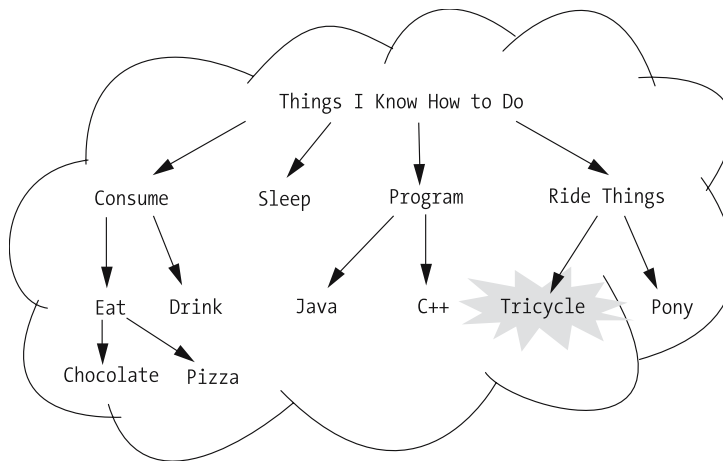


Figure 1-7. The human brain is adept at learning by building upon already-established abstractions.

This technique of comparing features to find an abstraction that is similar enough to be reused successfully is known as **pattern matching and reuse**. As we'll cover in Chapter 12, pattern reuse is an important technique for object-oriented software development, as well, because it spares us from having to reinvent the wheel with each new project. If we can reuse an abstraction or model from a previous project, we can focus on those aspects of the new project that differ from the old, gaining a tremendous amount of productivity in the process.

Inherent Challenges

Despite the fact that abstraction is such a natural process for human beings, developing an appropriate model for a software system is perhaps the most difficult aspect of software engineering, because

- There are an unlimited number of possibilities. Abstraction is to a certain extent in the eye of the beholder: several different observers working independently are almost guaranteed to arrive at different models. Whose is the best? *Passionate* arguments have ensued!
- To further complicate matters, there is virtually never only one “best” or “correct” model, only “better” or “worse” models relative to the problem to be solved. The same situation can be modeled in a variety of different, equally valid ways. When we get into actually doing some modeling in Part 2 of this book, we'll look at a number of valid alternative abstractions for our Student Registration System (SRS) case study that was presented at the end of the Introduction.
- Note, however, that there *is* such a thing as an *incorrect* model: namely, one that misrepresents the real-world situation (for example, modeling a person as having two different blood types).
- There is no acid test to determine if a model has adequately captured all of a user's requirements. The ultimate evidence of whether or not an abstraction was appropriate is in how successful the resultant software system turns out to be. We don't want to wait until the end of a project before finding out that we've gone astray. Because of this, it's critical that we learn ways of communicating our model concisely and unambiguously to the following people:
 - The intended future users of our application, so that they may provide a sanity check for our understanding of the problem to be solved before we embark upon software development
 - Our fellow software engineers, so that team members can share a common vision of what we're to build collaboratively

Despite all of these challenges, it's critical to get the up-front abstraction “right” before beginning to build a system. Fixing mistakes in the abstraction once a system is modeled, designed, coded, documented, and undergoing acceptance testing is much more costly (by orders of magnitude) than correcting the abstraction when it's still a gleam in the project team's eye. This isn't to imply that an abstraction should be rigid—quite the contrary! The art and science of object modeling, when properly applied, yields a model that is flexible enough to withstand a wide variety of functional changes. In addition, the special properties of objects further lend themselves to flexible software solutions, as you'll learn throughout the rest of the book. However, all things being equal, we'd like to harness this flexibility in expanding a system's capabilities over time, rather than in repairing mistakes.

What Does It Take to Be a Successful Object Modeler?

Coming up with an appropriate abstraction as the basis for a software system model requires

- **Insight into the problem domain:** Ideally, we'll be able to draw upon our own real-world experiences, such as former or current experience as a student, which will come in handy when determining the requirements for the SRS.
- **Creativity:** We must be able to think “outside the box,” in case the future users that we're interviewing have been immersed in the problem area for so long that they fail to see innovations that might be made.
- **Good listening skills:** These will come in handy as future users of the system describe how they do their jobs currently, or how they envision doing their jobs in the future, with the aid of the system that we're about to develop.
- **Good observational skills:** Actions often speak louder than words. Just by observing users going about their daily business, we may pick up an essential detail that they have neglected to mention because they do it so routinely that it has become a habit.

But all this isn't enough. We also need

- An organized **process** for determining what the abstraction should be. If we follow a proven checklist of steps for producing a model, then we greatly reduce the probability that we'll omit some important feature or neglect a critical requirement.
- A way to **communicate** the resultant model concisely and unambiguously to our fellow software developers and to the intended users of our application. While it's certainly possible to describe an abstraction in narrative text, a picture is worth a thousand words, and so the language with which we communicate a model is often a **graphical notation**. Throughout this book, we'll focus on the Unified Modeling Language (UML; see Figure 1-8) notation as our model communication language (you'll learn the basics of UML in Chapters 10 and 11). Think of a graphical model as a blueprint of the software application to be built.

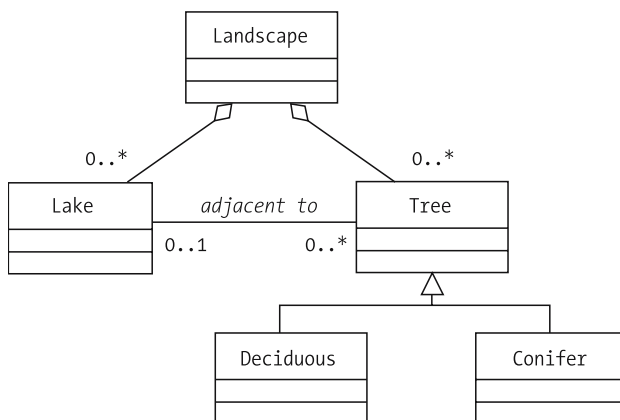


Figure 1-8. Describing a landscape in UML notation

- Ideally, we'll also have a **software tool** to help us automate the process of producing such a blueprint.

Part 2 of this book covers these three aspects of modeling—process, notation, and tool—in detail.

Summary

In this chapter, you've learned that

- Abstraction is a fundamental technique that people use to perceive the world.
- Developing an abstraction of the problem to be automated is a necessary first step of all software development.
- We naturally organize information into classification hierarchies based upon rules that we carefully structure, so that they are neither too general nor too restrictive.
- We often reuse abstractions when attempting to model a new concept.
- Producing an abstraction of a system to be built, known as a model, is in some senses second nature to us, and yet paradoxically is one of the hardest things that software developers have to do in the life cycle of an information systems project. It's also one of the most important.

IMPORTANT INFORMATION CONCERNING END-OF-CHAPTER EXERCISES

I've included exercises at the end of each chapter to assist you in your learning experience.

- Many of the exercises have been crafted as open-ended, thought-provoking questions suitable as homework assignments in an academic setting. Thus, there are no simple "one-size-fits-all" answers available for such exercises.
- Some exercises involve hands-on programming and are labeled as follows: [*Coding*] . . .
- You are encouraged to pose questions about the exercises, or about Java in general, via the Apress Forums online discussion group at <http://forums.apress.com>, where you get the benefit of insights from other readers, as well as from Apress authors/experts.

Exercises

1. Sketch a class hierarchy that relates all of the following classes in a reasonable manner:

Apple
Banana
Beef
Beverage
Cheese
Consumable
Dairy Product
Food
Fruit
Green Bean
Meat
Milk
Pork
Spinach
Vegetable

Justify your answer, noting in particular any challenges that you faced in doing so.

2. What aspects of a television set would be important to abstract from the perspective of
 - A consumer wishing to buy one?
 - An engineer responsible for designing one?
 - A retailer who sells them?
 - The manufacturer?
3. Select a problem area that you would like to model from an object-oriented perspective. Ideally, this will be a problem that you're actually going to be working on at your place of employment or that you otherwise have a keen interest in. Assume that you're going to write a program to automate some aspect of this problem area, and write a one-page overview of the requirements for this program, patterned after the SRS case study.

Make certain that your first paragraph summarizes the intent of the system, as the first paragraph in the SRS case study does. Also, emphasize the **functional requirements**—that is, those that a non-technical end user might state as to how the system should behave—and avoid stating **technical requirements**—for example, “This system must run on a Windows NT platform, and must use the TCP/IP protocol to . . .”

4. Read the case study for a Prescription Tracking System (PTS) in Appendix B. In your opinion, how effective is this case study as an abstraction? Are there details that you think could have been omitted or missing details that you think would have been important to include? If you had an opportunity to interview the intended users of the PTS, what additional questions might you ask them to better refine this abstraction?
-



Some Java Basics

The goals for this book are not only to teach you about objects and object modeling, but also to illustrate how objects translate into a working software application. So, before diving into the basics of objects in Chapter 3, we'll spend some time getting comfortable with Java fundamentals, as this is the programming language used to illustrate object concepts as they are introduced throughout the book.

Objects are “language neutral,” and so what you'll learn conceptually about objects in Part 1 of this book, and about object modeling in Part 2, could apply equally well to Java, or C++, or C#, or an as-yet-to-be-invented object-oriented (OO) language. I've chosen Java specifically for the many advantages that this elegant OO programming language affords us as software developers, which we'll explore in this chapter.

In this chapter, you'll also learn about

- Primitive Java types, operators on those types, and expressions formed with those types
- The anatomy of a simple Java program
- The mechanics of compiling and running such programs
- Java's **block-structured** nature
- Various types of Java **expressions**
- Loops and other flow control structures
- Printing messages to the command window from which a program was launched, which is especially useful for testing code as it evolves
- Elements of Java programming style

If you're a proficient C, C++, or C# programmer, you'll find much of Java syntax to be very familiar, and you should be able to breeze through this chapter fairly quickly.

Why Java?

After learning what makes objects “tick” in Part 1 of the book and how to model an application to take advantage of objects in Part 2, you'll be ready for the grand finale: rendering an object model in code to produce a working Student Registration System (SRS) application in Part 3. As mentioned earlier, we *could* walk through building the SRS using any OO programming language. Why might we want to use Java? Read on, and you'll quickly see why!

Java Is Architecture Neutral

To execute a program written in a conventionally compiled language like C or C++, the source code must first be compiled into an executable form known as **binary code** or **machine code**. Binary code, in essence, is a pattern of 1s and 0s understandable by the underlying **hardware architecture** of the computer on which the program is intended to run.

Even if original C or C++ source code is written to be **platform independent**—that is, the program does not take advantage of any platform-specific language extensions such as a specific type of file access or graphical user interface (GUI) manipulation—the resultant executable version will nonetheless still be tied to a particular platform's architecture and can therefore be run on only that architecture. That is, a version of the program compiled for a Sun Solaris workstation will not run on a Windows PC, a version compiled for a Windows PC will not run on a Linux machine, and so forth. This concept is depicted in Figure 2-1.

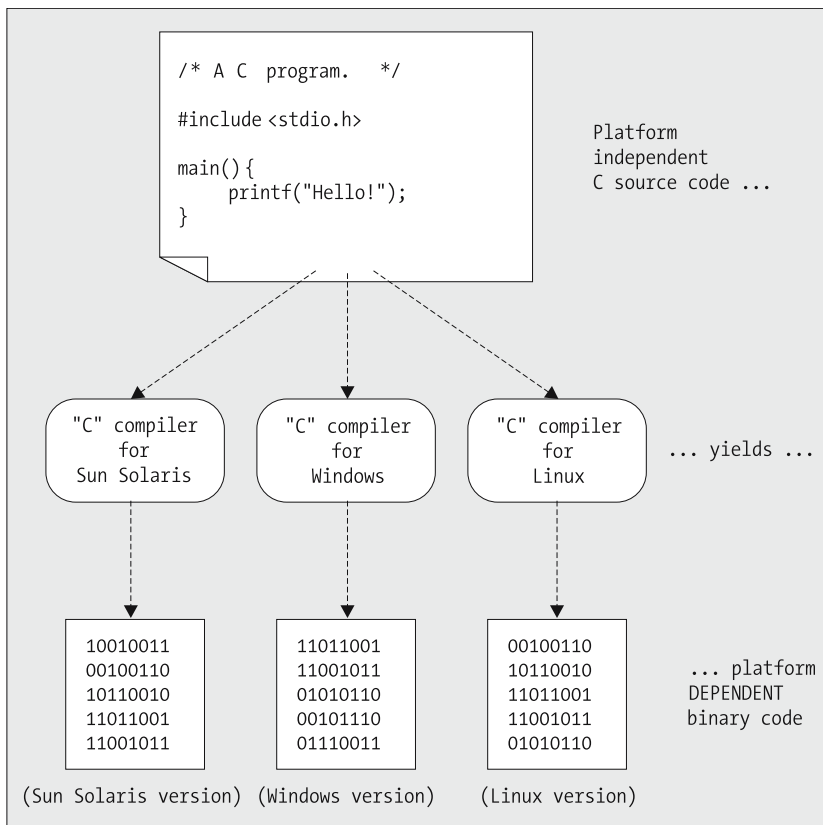


Figure 2-1. *Conventionally compiled languages yield platform-dependent executable programs.*

In contrast, Java source code is not compiled for a particular platform, but rather into a special intermediate format known as **bytecode**, which is said to be both platform independent and **architecture neutral**. That is, no matter whether a Java program is compiled under Sun Solaris, Windows, Linux, or any other operating system for which a Java compiler is available, the resultant bytecode turns out to be the same and hence can be run on any computer for which a (platform-*specific*) Java Virtual Machine (JVM) is available. This is illustrated in Figure 2-2.

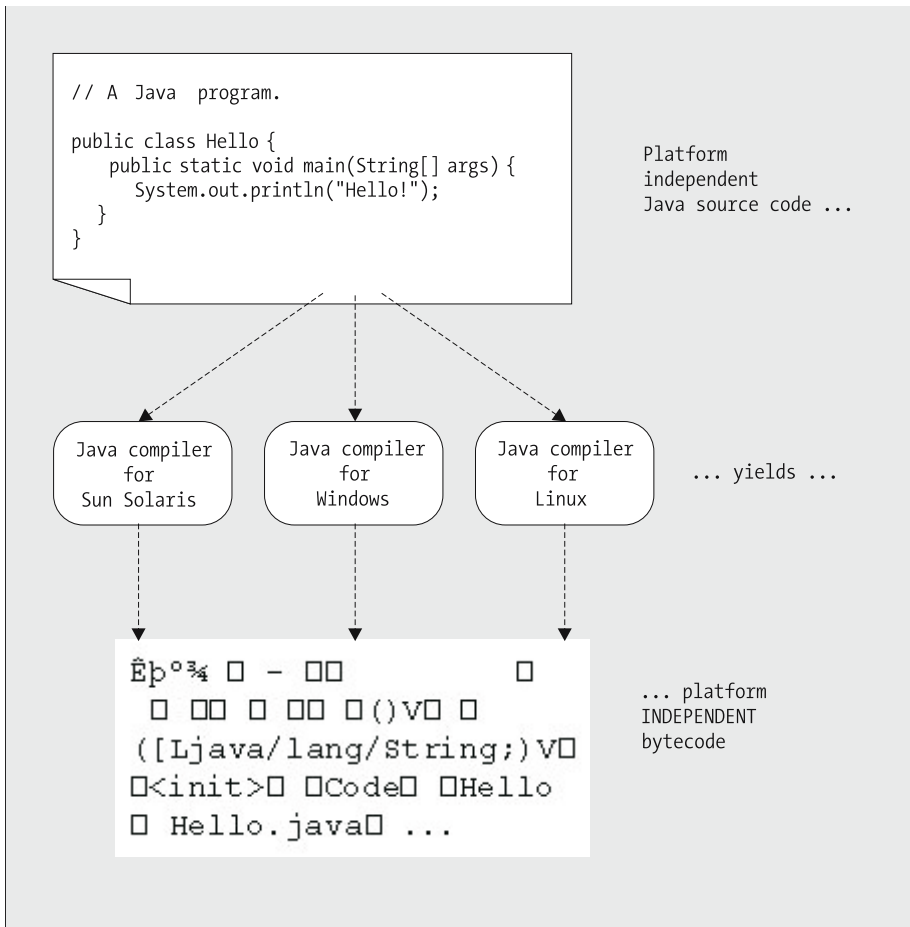


Figure 2-2. *The Java compiler yields platform-independent bytecode.*

The JVM is a special piece of software that knows how to **interpret** and execute Java bytecode. That is, instead of a Java program running directly under the control of the operating system the way traditionally compiled programs do, the JVM itself runs under direct control of the operating system, and our Java program in turn runs under control of the JVM, as illustrated in Figure 2-3. The JVM in essence serves as a translator, translating the universal Java bytecode “language” into the machine code “language” that a particular computer can understand, in the same way that a human interpreter facilitates a discussion between someone speaking German and someone speaking Japanese by translating their statements as they converse.

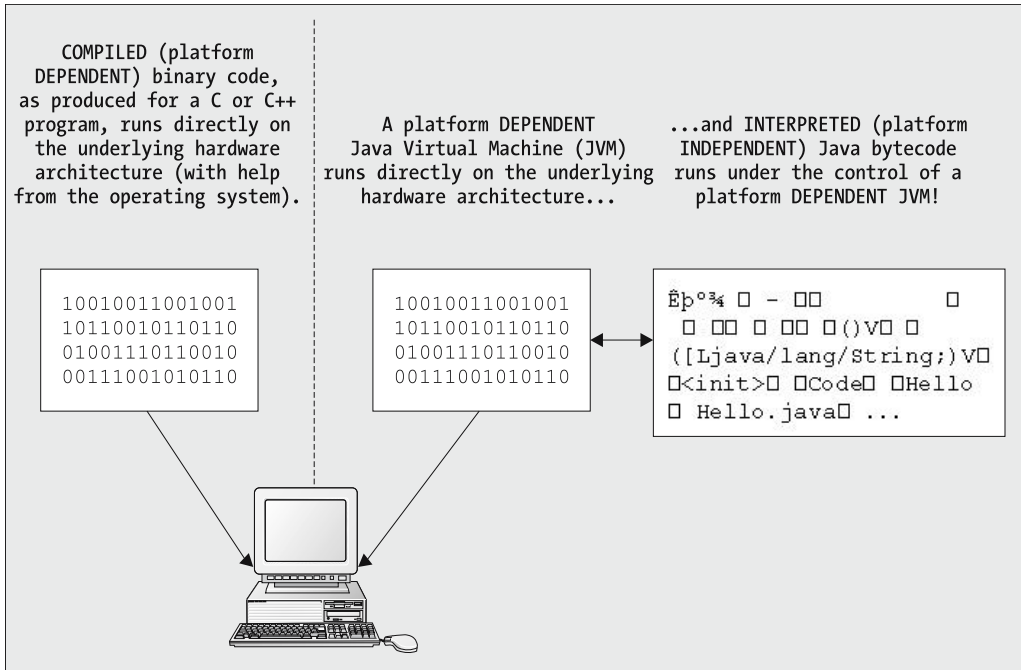


Figure 2-3. A platform-dependent JVM is used to execute platform-independent bytecode.

The interpreted nature of the Java language tends to make it a *tiny* bit slower, in general, than compiled languages because there is an extra processing layer involved when an application executes, as illustrated in Figure 2-3. For traditional information systems applications that involve a human user in the loop, however, the difference in speed is imperceptible; other factors, such as the speed of the network (in the case of distributed applications), the speed of a DBMS server (if a database is used), and especially human “think time” while responding to an application’s user interface, can cause any JVM response time delays to pale by comparison.

As long as you have the appropriate JVM installed on a given target platform, you can transfer Java bytecode from one platform to another without recompiling the original Java source code, and it will still be able to run. That is, bytecode is transferable across platforms, as illustrated in Figure 2-4.

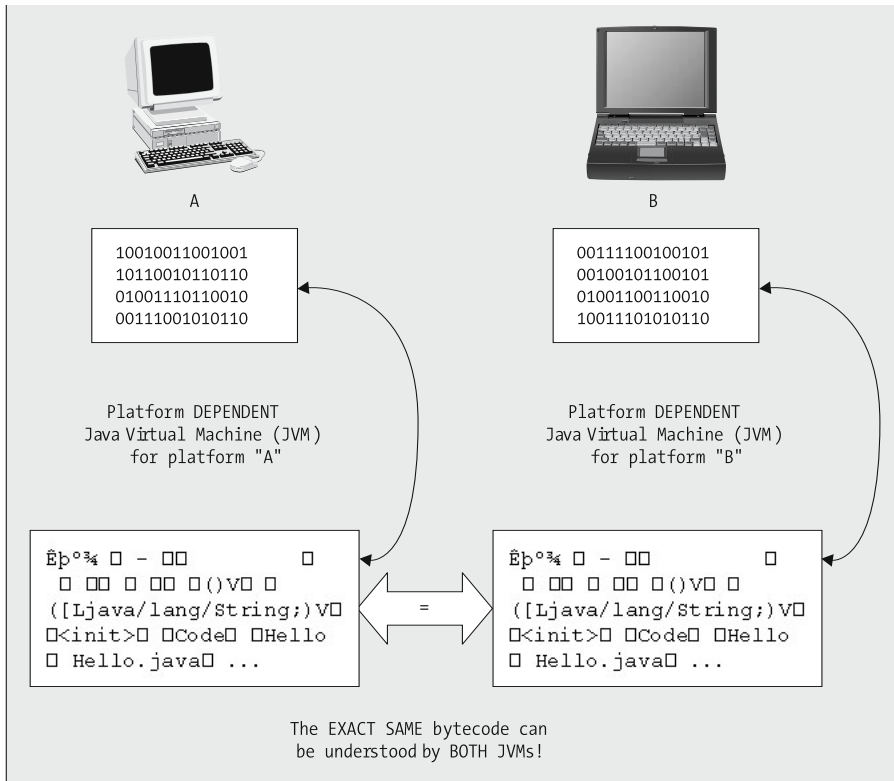


Figure 2-4. The exact same bytecode is understood by JVMs on two different platforms.

MIGRATING BYTECODE

There are two important caveats to keep in mind when migrating bytecode from one machine to another. The first involves migrating across different **versions** of the Java language, even if on the same platform. Java bytecode is, in theory, **forward compatible** with newer versions of the JVM, meaning that if a Java program is compiled with one version of the Java language—say, version 1.3.1—it will run correctly under a newer version of the JVM—say, version 1.4.2. However, there can be slight differences in the way an application runs under a newer version of the JVM, due to nothing more than perhaps that a minor bug in an earlier version of the Java language has been corrected.

Even when migrating to the **same** version of Java across **different** hardware architectures, however, minor incompatibilities can arise. Sun Microsystems is responsible for releasing new versions of Java for only a handful of platforms—in particular, newer “flavors” of Windows, Linux, and Sun Solaris (Unix). It’s left up to other hardware vendors to provide compatible JVMs for their respective platforms (e.g., Apple Macintosh and SGI IRIX [Unix]). I’ve encountered situations where a particular vendor releases a version of Java that has the same version number as a Sun release of Java, but where the features of that vendor’s version don’t quite measure up to the similarly numbered version from Sun.

The bottom line is that it’s best to recompile Java source code for a given target platform before transferring the resultant bytecode to that platform, if you have that luxury.

Java Provides “One-Stop Shopping”

With most conventional programming languages, the core language doesn't automatically provide everything that you'll need to build an industrial-strength application, with a GUI and access to a database management system. For these capabilities, you must typically integrate platform-specific (and often, *vendor*-specific) libraries into your application, as illustrated in Figure 2-5.

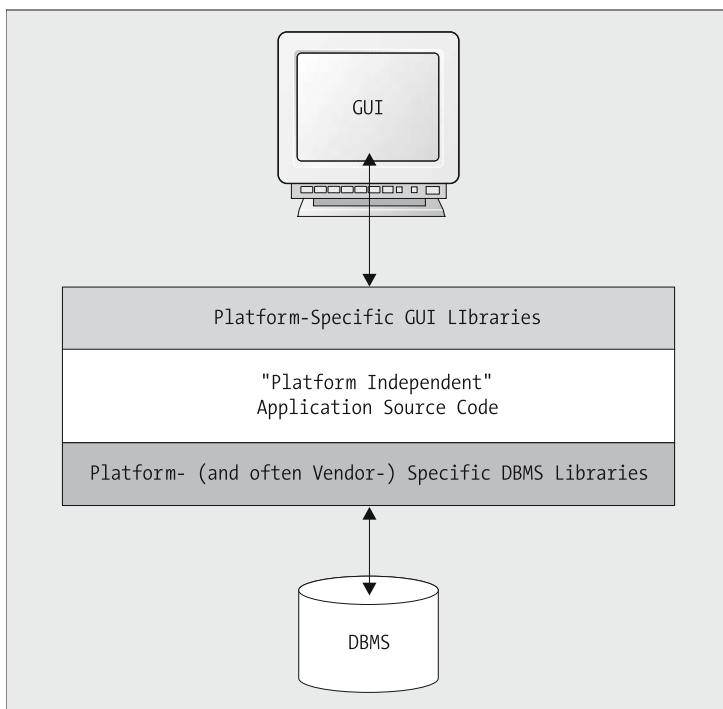


Figure 2-5. Integrating platform-specific libraries into an otherwise platform-independent application

The inclusion of such platform- and vendor-specific library calls hinders application migration from one platform to another, because not all library calls for a given platform *X* necessarily *have* an equivalent for a target platform *Y*—for example, not all Microsoft ActiveX controls (used to build GUIs for Windows computers) have an equivalent in the Motif library of GUI components (used to build GUIs for Unix machines), and vice versa.

Worse yet, the boundary between the platform-*independent* and platform-/vendor-*dependent* code is not really as clean as is depicted in Figure 2-5. Calls to these libraries are often scattered throughout an application, as illustrated in Figure 2-6, making the chore of migrating an application from one platform to another troublesome at best and prohibitively difficult in the worst cases.

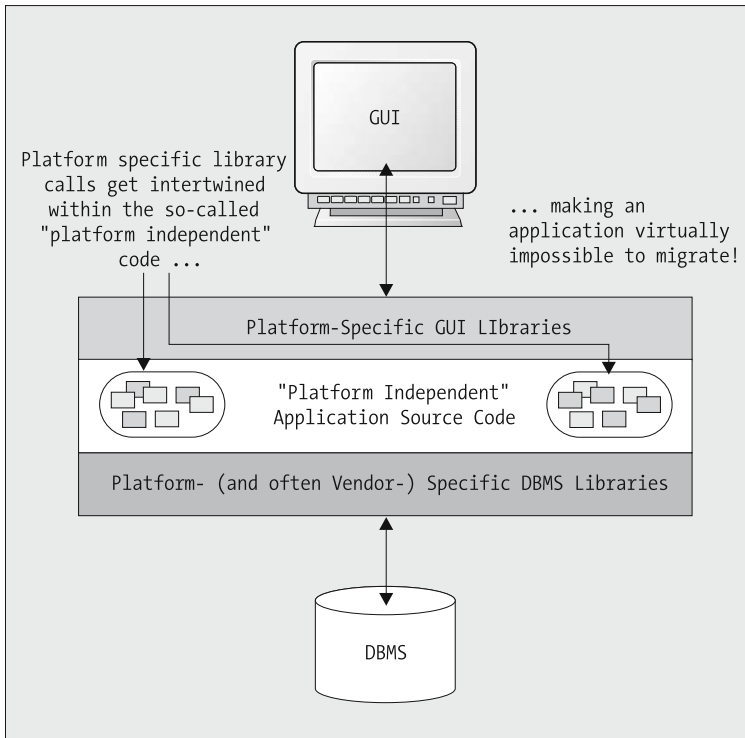


Figure 2-6. *The boundaries between platform-dependent and platform-independent code are unclear.*

If the boundary between platform-independent and platform-dependent components of an application is particularly messy, it's sometimes the lesser of two evils to rewrite the entire application from scratch rather than to attempt to migrate it.

There are some special third-party programs available called **emulators** that will simulate one operating system under another, thus enabling us (in theory) to execute a platform-dependent application on another incompatible platform without any modification. However, not all applications will necessarily run properly under an emulator, and those that do typically perform sluggishly.

In contrast, the Java language provides an extensive set of **application programming interfaces (APIs)** that provide a consistent, **platform-independent** means of accessing all underlying operating system functions, including GUI rendering and database management system (DBMS) access. We'll cover the following Java APIs later, in Part 3 of the book:

- `java.io`: Used for file system access (see Chapter 15)
- `java.sql`: The JDBC API, used for communicating with relational databases in a vendor-independent fashion (see Chapter 15)
- `java.awt`: The Abstract Windowing Toolkit, used for GUI development (see Chapter 16)
- `javax.swing`: Swing components, also used for GUI development (see Chapter 16)

And there are *many* more! If these built-in Java libraries are used in developing 100% pure Java applications as illustrated in Figure 2-7, the resultant Java code is truly portable across platforms and vendors. So, to migrate a *Java* application from Windows XP to Linux, for example, there's no need to rip out code (or your hair!): simply transfer the bytecode for the application to the new platform, and as long as the new host platform has the appropriate version of a platform-specific JVM installed, then you are good to go.



Figure 2-7. A 100% pure Java application is platform-independent from GUI to DBMS.

In Part 3 of the book, we'll discuss two *critically important* design techniques—**model-data access layer separation** and **model-view separation**—which will further ensure the vendor- and platform- flexibility of our applications.

Java Is Object-Oriented from the Ground Up

Before newer OO languages like Java arrived on the scene, one of the most widely used OO languages was C++, which is actually an object-oriented extension of the non-OO language C. As such, C++ provides a lot of “back doors” that make it very easy to write decidedly un-OO code. In fact, many proficient C programmers transitioned to C++ as a better C without properly learning how to design an OO application, and hence wound up using C++ for the most part as a procedural (non-OO) language.

If you're an experienced C++ programmer wishing to know more about the differences between C++ and Java, please refer to Appendix E.

In contrast, Java is object oriented to its very core. As you will learn in more detail in the chapters that follow, just about *everything* in Java is an object:

- All data, with the exception of a few primitive types, is rendered as objects.
- All of the GUI building blocks—windows, buttons, text input fields, scroll bars, lists, menus, and so on—are objects.

- All functions are associated with objects and are known as **methods**—there can be no “free-floating” functions as there are in C/C++.
- Even the main function used to launch an application (in Java it's called the **main method**) no longer stands alone, but is instead bundled within a **class**, the reasons for which we'll explore in depth in chapters to come.

Because of this, Java lends itself particularly well to writing applications that uphold the OO paradigm. Yet, as I pointed out in the introduction to this book, merely using such an OO language does not *guarantee* that the applications you produce will be *true* to this paradigm! You must be knowledgeable both in how to design an application from the ground up to make the best use of objects and in how to apply the language correctly, which are the primary intents of this book.

Practice Makes Perfect

The Java language designers at Sun Microsystems were able to draw upon the lessons learned from other OO programming languages that preceded it. They borrowed the best features of C++, Eiffel, Ada, and Smalltalk, and then added some capabilities and features not found in those languages. Conversely, features that had proven to be most troublesome in those earlier languages were eliminated.

Also, as of version 5.0 (aka 1.5) of Java, several features of a *newer* language, Microsoft's C#, were retrofitted into Java.

This isn't to say that Java is a “perfect” language (no language is!), but simply that it has made some significant improvements over many of the languages that have preceded it.

Java Is an Open Standard

What significance does the fact that Java is an open standard hold for us as software developers? For one thing, all of the source code behind all of the built-in Java libraries is available for us to study and use as a basis for our own software design.

Also, the IT industry as a whole has been quick to embrace Java because no third-party vendors have been excluded, as is often the case with a proprietary/“closed” technology. Quite the contrary: Sun Microsystems has encouraged other vendors to adopt the Java standard and to incorporate Java capability in their products by allowing vendors to participate in the advancement of Java technology specifications through Sun's **Java Community Process**.

And, if the preceding reasons still aren't compelling enough to convince you of the merits of Java . . .

Java Is Free!

Sun Microsystems has ensured that Java enjoys widespread adoption, in part, by making the language and all of the basic tools necessary to develop Java applications *free*! Appendix C provides information on what you'll need to do to download the **Java 2 Software Development Kit (SDK)** from Sun's website (<http://java.sun.com>) to get started with Java development. (I provide instructions for how to compile and run Java programs a bit later in this chapter.)

The rest of this chapter introduces some elemental Java syntax, which we'll continue to build upon in subsequent chapters.

A Reminder Regarding Pseudocode vs. Real Java Code

I occasionally use little bits of pseudocode in the code examples throughout Parts 1 and 2 of this book to hide irrelevant logic details. To make it clear as to when I'm using pseudocode rather than real code, I use *italic* rather than regular SansMono condensed font.

This is real Java syntax:

```
for (int i = 0; i <= 10; i++) {
```

This is pseudocode:

```
compute the grade for the ith Student  
}
```

I'll remind you of this fact a few more times, so that you don't forget and accidentally try to type in and compile pseudocode somewhere along the way.

Anatomy of a Simple Java Program

Figure 2-8 shows one of the simplest of all Java applications.

File named SimpleProgram.java contains:

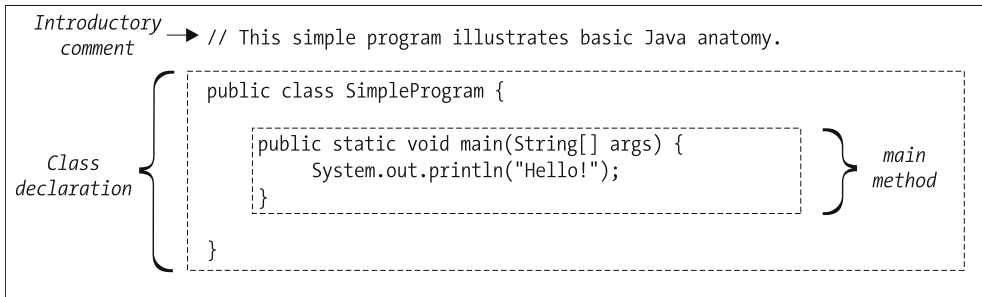


Figure 2-8. *Anatomy of a simple Java program*

Let's go over the key elements of our simple program.

Comments

The first thing we see in our simple Java program is an introductory comment:

```
// This simple program illustrates some basic Java syntax.
```

Java supports three different comment styles: traditional, end-of-line, and Java documentation comments.

Traditional Comments

Java **traditional comments** derive from the C language and begin with a forward slash followed by an asterisk (/*) and end with an asterisk followed by a forward slash (*/). Everything enclosed between these delimiters is treated as a comment and is therefore ignored by the Java compiler, no matter how many lines the comment spans.

```
/* This is a traditional (C-style) comment. */
```

```
/* This is a multiline traditional comment. This is a handy way to temporarily
   comment out entire sections of code without having to delete them.
```

```
   From the time that the compiler encounters the first "slash asterisk"
   above, it doesn't care what we type here; even legitimate lines of code,
   as shown below, are treated as comment lines and thus ignored by the
   compiler until the first "asterisk slash" combination is encountered.
```

```
x = y + z;
a = b / c;
j = s + c + f;
*/
```

```
/* We often use leading asterisks on the second through last line of a traditional
   * comment simply for cosmetic reasons, so that the comment is more visually
   * distinct; but, these extra asterisks are strictly cosmetic - only the
   * initial "slash asterisk" and the final "asterisk slash" are noted by the
   * compiler as having any significance.
   */
```

Note that we can't *nest* block comments—that is, the following will *not* compile:

```
/* This starts a comment ...
x = 3;
/* Whoops! We are mistakenly trying to nest a SECOND comment
before terminating the FIRST!
This is going to cause us compilation problems, because the
compiler is going to IGNORE the start of this second/inner comment -- we're IN
a comment already, after all! -- and so as soon as we try to terminate
this SECOND/inner comment, the compiler will think that we've terminated the
FIRST/outer comment instead ... */
z = 2;
// The compiler will "complain" on the next line.
*/
```

When the compiler reaches what we intended to be the terminating */ of the “outer” comment in the last line of the preceding code example, the following two compilation errors will be reported:

```
illegal start of expression
   */
   ^
```

and

```
';' expected
  */
  ^
```

End-of-Line Comments

The second type of Java comment derives from C++ and is known as an **end-of-line comment**. We use a double slash (//) to note the beginning of a comment that automatically ends when the end of the line is reached, as shown here:

```
x = y + z;    // text of comment continues through to the end of the line ==>
a = b / c;
```

```
// Here's a BLOCK of sequential end-of-line comments.
// This serves as an alternative to using traditional comments
// (/* ... */) and is preferred by many Java programmers.
```

```
m = n * p;
```

Java Documentation Comments

The third and final type of Java comment, **Java documentation comments** (aka **Javadoc comments**), can be parsed from source code files by a special javadoc command-line utility program (which comes standard with the Java SDK) and used to automatically generate HTML documentation for an application.

We'll defer an in-depth look at Javadoc comments until Chapter 13.

The Class Declaration

Next comes a **class wrapper**—more properly termed a **class declaration**—of the form

```
public class ClassName {
    ...
}
```

For example:

```
public class SimpleProgram {
    ...
}
```

where braces { ... } enclose the **class body** that includes the main logic of the program along with other optional building blocks of a class.

In subsequent chapters, you'll learn all about the significance of classes in an OO programming language. For now, simply note that the symbols `public` and `class` are two of Java's **keywords**—that is, symbols reserved for specific uses within the Java language—whereas `SimpleProgram` is a name/symbol that I've invented.

The main Method

Within the `SimpleProgram` class declaration, we find the starting point for the program, called the **main method** in Java. The main method serves as the entry point for a Java program. When we execute a Java program by interpreting its bytecode with an instance of the JVM, the JVM calls the main method to jump-start our application.

With trivial applications such as the `SimpleProgram` example, all of the program's logic can be contained within this single main method. For more complex applications, on the other hand, the main method can't possibly contain all of the logic for the entire system. You'll learn how to construct an application that transcends the boundaries of the main method, involving multiple Java source code files, a bit later in the book.

The first line of the method, shown here

```
public static void main(String[] args) {
```

defines what is known as the main method's **method header**, and must appear exactly as shown (with one minor exception that I'll explain in Chapter 13 having to do with optionally receiving arguments from the command line).

Our main method's **method body**, enclosed in braces { ... }, consists of a single statement:

```
System.out.println("Hello!");
```

which prints the message

```
Hello!
```

to the (DOS/Solaris/Linux) command window from which our program is launched. We'll examine this statement's syntax further in a bit, but for now, note the use of a semicolon (;) at the end of the statement. As in C and C++, semicolons are placed at the end of all individual Java statements. Braces { ... }, in turn, delimit **blocks** of code, the significance of which I'll discuss in more detail a bit later in this chapter.

Other things that we'd typically do inside of the main method of a more elaborate program include declaring variables, creating objects, and calling other methods.

Now that we've looked at the anatomy of a simple Java program, let's discuss how such a program is compiled and executed.

Please refer to Appendix C for details on how to get the Java 2 Software Development Kit (SDK) properly installed on your computer before proceeding.

The “Mechanics” of Java

Once we have entered our program logic, as text, into a file, using either a simple text editor (such as Windows Notepad or vi) or a Java Integrated Development Environment (IDE) tool, we must first compile the source code into bytecode before we can execute it.

For folks just getting started with Java programming, I advocate using a simple text editor at first so that you aren't distracted by the bells and whistles of a specific IDE, and so that the IDE doesn't do so much work for you that you don't really learn what's going on at the most basic level. If you like working with IDEs, however, I recommend starting out with a very inexpensive tool called **TextPad**, available at <http://www.textpad.com>.

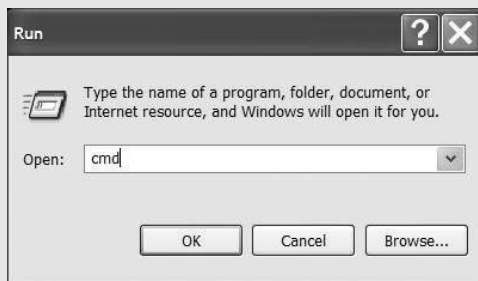
For folks who are proficient with vi (a Unix/Linux editor), note that there is a Windows/DOS-compatible version called **vim** that is available at no cost from <http://www.vim.org>—specifically, it can be downloaded from <http://www.vim.org/download.php>.

Once you've mastered the Java language, you can “graduate” to a more complex IDE if you wish.

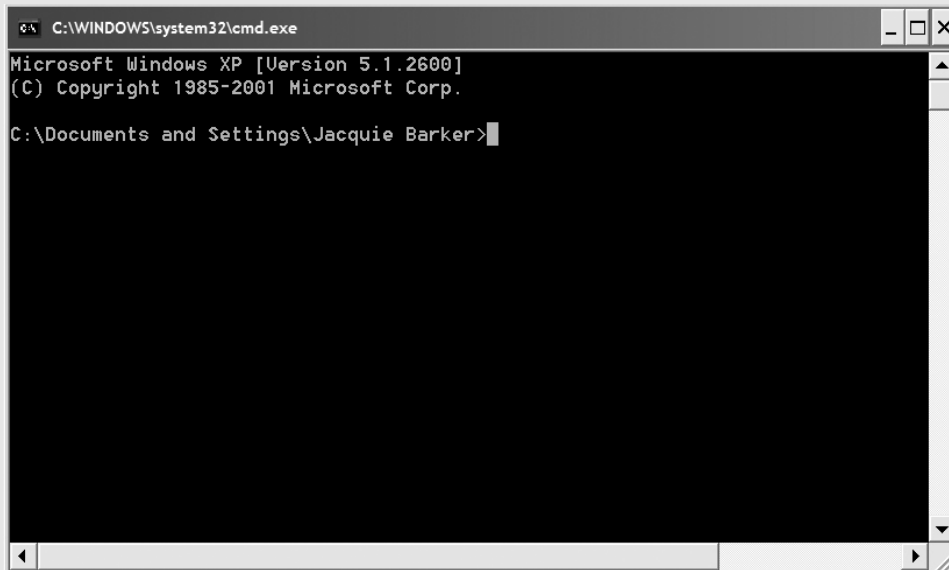
The simplest way to compile and execute a Java program is via command-line commands, which are the same whether we're developing on a Sun Solaris, Linux, or Windows computer.

USING THE COMMAND PROMPT

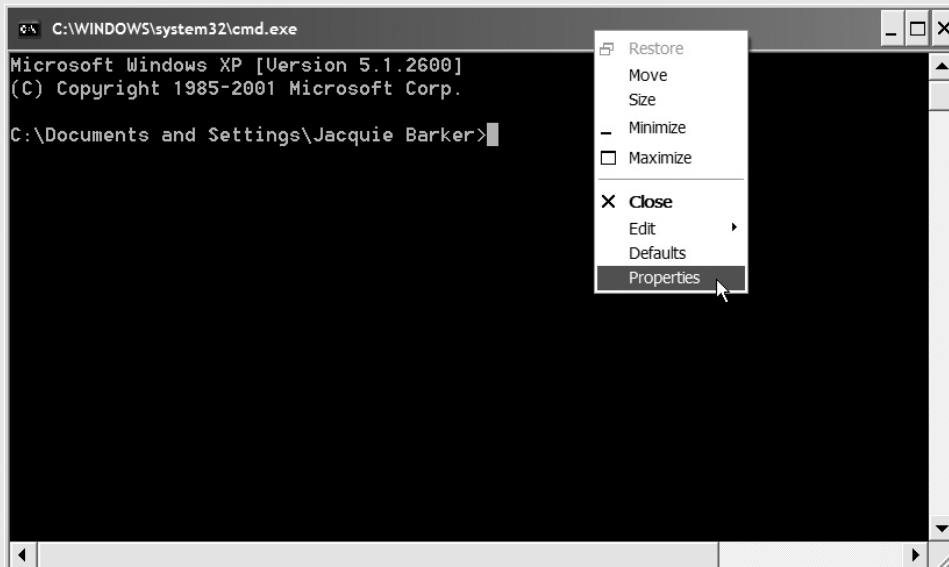
In the case of Windows, open a **Command Prompt** window by selecting the Start ► Run menu, then type `cmd` in the dialog box that appears, as shown in the following image; then click OK.



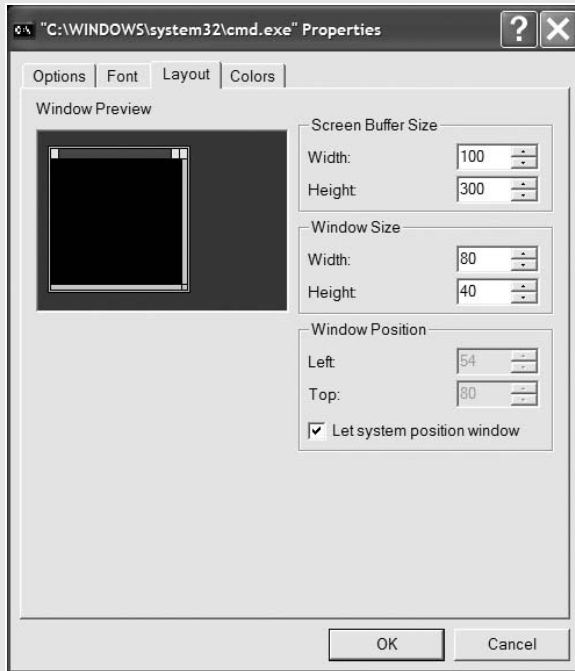
A sample Command Prompt window is shown in the following image.



To expand the capacity of the Command Prompt window and/or to resize it, right-click the blue header bar of the window and choose Properties from the pop-up menu that appears, as shown in the following image.



On the Layout panel of the dialog box that appears (as shown in the following image for a Windows XP computer), adjust the Screen Buffer Size and Window Size options as indicated.



Compiling Java Source Code into Bytecode

To compile Java source code from the command line, we use the `cd` command, as necessary, to navigate into the working directory where our source code resides. We then type the following command:

```
javac sourcecode_filename
```

for example:

```
javac SimpleProgram.java
```

to compile it.

If there were more than one `.java` source code file in the same directory, we could either list the names of the files to be compiled, separated by spaces:

```
javac Foo.java Bar.java Another.java
```

or use the wildcard character (`*`), for example:

```
javac *.java
```

to compile multiple files at the same time.

If all goes well—namely, if no compiler errors arise—then a bytecode file by the name of `SimpleProgram.class` will appear in the same directory where the `SimpleProgram.java` source code file resides. If compiler errors do arise, on the other hand, we of course must correct our source code and attempt to recompile it.

Please refer to the section entitled “Common Compilation Errors” in Appendix C for a list of some of the common compilation errors and how to resolve them.

Executing Bytecode

Once a program has been successfully compiled, we execute the bytecode version via the command

```
java bytecode_filename      (note that we OMIT the .class suffix)
```

for example:

```
java SimpleProgram
```

Note that it's important to *omit* the `.class` suffix of the bytecode file name (which is called `SimpleProgram.class` in this case).

By default, the JVM will look in your default working directory along with the “home” directory where the Java language has been installed on your computer system for such bytecode files. If the JVM finds the specified bytecode file, it executes its `main` method, and your program is off and running!

If for some reason the bytecode you are trying to execute is in neither of these two default locations, you must inform the JVM of additional directories in which to search. You can do so by specifying a list of directories (separated by semicolons [`;`] under Windows, or by colons [`:`] under Solaris and Linux) after the `-cp` flag on the `java` command as follows:

```
java -cp list_of_directory_names_to_be_searched bytecode_filename
```

For example, on DOS/Windows:

```
java -cp C:\home\javastuff;D:\reference\workingdir;S:\foo\bar\files SimpleProgram
```

or on Solaris/Linux:

```
java -cp /barkerj/work:/java/examples/ex1 SimpleProgram
```

At a minimum, we typically want the JVM to search our *current working directory* for bytecode files. With version 1.4.x of Java and *later*, this happens automatically, as discussed earlier; for versions 1.3.x of Java and *earlier*, we must explicitly specify a single period (`.`), which is (DOS/Linux/Solaris) shorthand for “the current working directory”, as a classpath entry; for example, for DOS/Windows:

```
java -cp . SimpleProgram
```

or

```
java -cp .;C:\home\javastuff;D:\reference\workingdir SimpleProgram
```

Please refer to the section entitled “Common Run-Time Errors” in Appendix C for a list of some common run-time errors and how to resolve them.

A Behind-the-Scenes Look at the JVM

Let's take a more in-depth look at what is actually happening behind the scenes when we type the command

```
java SimpleProgram
```

to run our program.

- The `java` command causes the JVM to be launched. Recall from an earlier discussion in this chapter that the JVM is a bytecode interpreter.
- The JVM in turn looks for the bytecode file that we've named in the command (`SimpleProgram.class` in this case), searching the directories enumerated by the `-cp` flag, if provided, or in the default working directory if `-cp` isn't used.
- Assuming that the JVM does indeed find the appropriate bytecode file, it loads the bytecode into its memory (conceptually, the bytecode serves as a "plug-in" for the JVM).
- Next, the JVM searches the bytecode that it has just loaded for the presence of the official `main` method header. If this method is found, the JVM executes it, and our program is off and running!

Now that we've looked at the mechanics of compiling and running Java programs, let's explore some of the basic syntax features of Java in more detail.

Primitive Types

Java is said to be a **strongly typed** programming language, in that when a variable is declared, its type must also be declared. Among other things, declaring a variable's type tells the compiler how much memory to allocate for the variable at run time, and also constrains the context(s) in which that variable may subsequently be used in our program.

The Java language defines eight **primitive types** (all eight of these type names are Java keywords), as follows.

Four types of ***integer*** numeric data:

- `byte`: 8-bit unsigned integer
- `short`: 16-bit signed integer
- `int`: 32-bit signed integer
- `long`: 64-bit signed integer

Two ***floating-point*** numeric types:

- `float`: 32-bit single-precision floating point
- `double`: 64-bit double-precision floating point

Plus two additional primitive types:

- **char**: A single character, stored using 16-bit Unicode encoding (versus 8-bit ASCII encoding), that enables Java to handle a wide range of international character sets.
- **boolean**: A variable that may only assume one of two values: `true` or `false` (both of these values are reserved words in Java). Boolean variables are often used as flags to signal whether or not some code should be conditionally performed, as in the following code snippet:

```
boolean error = false; // Initialize the flag.
// ...

// Later in the program (pseudocode):
if (some error situation arises) {
    // Set the flag to true to signal that an error has occurred.
    error = true;
}
// ...

// Still later in the program:
// Test the flag's value.
if (error == true) {
    // Pseudocode.
    take corrective action ...
}
```

We'll talk specifically about the syntax of the `if` statement, one of several different kinds of Java flow-control statements, a bit later in this chapter.

An important reminder: If you wish to attempt to compile any of the Java code snippets that you come across throughout the book, remember that (a) pseudocode (*italicized*) won't compile, and (b) all code must, at a minimum, be enclosed within a `main` method, which in turn must be enclosed within a `class` declaration, as was illustrated in Figure 2-8.

Variables

Before a variable can be used in a Java program, the type and name of the variable must be **declared** to the Java compiler, for example:

```
int count;
```

Assigning a value to a variable is accomplished by using the Java **assignment operator**, `=`. An assignment statement consists of a (previously declared) variable name to the left of the `=`, and an expression that evaluates to the appropriate type to the right of the `=` (we'll cover various types of Java expressions later in the chapter). For example:

```
int count = 1;

total = total + 4.0; // Here, we assume that total was declared to be a double
                    // variable earlier in the program.

price = cost + (a + b)/length; // We once again assume that all variables were
                               // properly declared earlier in the program.
```

An initial value can be supplied when a variable is first declared:

```
int count = 3;

or a variable can be declared in one statement, then assigned a value in a separate statement
later on in the program:

double total;
// intervening code ... details omitted
total = total + 4.0;
```

A value can be assigned to a boolean variable using the true or false literals:

```
boolean finished;
// ...
finished = true;
```

A literal value may be assigned to a variable of type char by enclosing the value (a single Unicode character) in *single* quotes:

```
char c = 'A';
```

The use of *double* quotes ("...") is reserved for assigning literal values to String variables, a distinct type discussed later in this chapter. The following would not compile in Java:

```
char c = "A"; // We must use single quotes when assigning values to char variables.
```

Variable Naming Conventions

When discussing Java variable names, there are two aspects to consider:

- First, is a particular name deemed *valid* by the Java compiler?
- Second, does a particular *valid* name adhere to the naming *convention* that has been adopted by the OO programming community across all languages?

Valid variable names in Java must start with either an alphabetic character, an underscore, or a dollar sign (whose use is discouraged, since it is used by the compiler when generating code), and may contain any of these characters plus numeric digits. No other characters are allowed in variable names.

The following are all valid variable names in Java:

```
int simple;           // starts with alphabetic character
int _under;          // starts with underscore
int more$money_is_2much; // may contain dollar signs, and/or underscores, and/or
                        // digits, and/or alphabetic characters
```

while these are invalid:

```
int 1bad;             // inappropriate starting character
int number#sign;      // contains invalid character
int foo-bar;          // ditto
int plus+sign;        // ditto
int x@y;              // ditto
int dot.notation;     // ditto
```

That being said, the **convention** that is observed throughout the OO programming community is to form variable names using primarily alphabetic characters, avoiding the use of underscores, and furthermore to adhere to a style known as **camel casing**. With camel casing, the first letter of a variable name is in **lowercase**, the **first** letter of each subsequent concatenated word in the variable name is in **uppercase**, and the rest of the characters are in **lowercase**. All of the following variable names are both valid and conventional:

```
int grade;
double averageGrade;
String myPetRat;
boolean weAreFinished;
```

Recall that, as mentioned earlier, Java keywords can't be used as variable names. The following won't compile, because `public` is a Java keyword:

```
int public;
```

In fact, the compiler would generate the following **two** error messages:

```
not a statement
int public;
^
```

```
',' expected
int public;
^
```

Variable Initialization

In Java, variables aren't necessarily assigned an initial value when they're declared, but all variables **must** be assigned a value before the variable's value is **used** in an assignment statement. For example, in the following code snippet, two `int` (eger) variables are declared; an initial value is explicitly assigned to variable `foo`, but not to variable `bar`. A subsequent attempt to add the two variables' values together results in a compiler error:

```
int foo;
int bar;
// We're explicitly initializing foo, but not bar.
foo = 3;
foo = foo + bar;    // This line won't compile.
```

The following compiler error would arise on the last line of code:

```
variable bar might not have been initialized
foo = foo + bar;
      ^
```

To correct this error, we would need to assign an explicit value to bar, as well as foo, before using them in the addition expression:

```
int foo;
int bar;
foo = 3;
// We're now initializing BOTH variables explicitly.
bar = 7;
foo = foo + bar;    // This line will now compile properly.
```

In Chapter 13 you'll learn that the rules of automatic initialization are somewhat different when dealing with the “inner workings” of objects.

The String Type

We'll look at one more important Java type in this chapter: the String type. A String represents a sequence of zero or more Unicode characters.

The symbol `String` starts with a capital “S,” whereas the names of primitive types are expressed in all lowercase: `int`, `float`, `boolean`, etc. This capitalization difference is deliberate and mandatory—`string` (lowercase) won't work as a type:

```
string s = "foo"; // This won't compile.
```

Here is the error message:

```
cannot find symbol
symbol:   string
```

(I'll explain the significance of the capitalization of `String` as a type in due time.)

There are several ways to create and initialize a String variable. The easiest and most commonly used way is to declare a variable of type `String` and to assign the variable a value using a **string literal**. A string literal is any text enclosed in *double* quotes, even if it consists of only a *single character*.

```
String name = "Steve";    // Note the use of double quotes, regardless of the
String shortString = "A"; // length, when we're assigning a literal value
                          // to a String variable.
```

Two commonly used approaches for initializing a `String` variable with a temporary placeholder value are as follows:

- Assigning an empty string, represented by two consecutive double quote marks:

```
String s = "";
```

- Assigning the value `null`, which is a Java keyword that is used to signal that a `String` has not yet been assigned a “real” value (as you’ll learn later in the book, we’ll use the `null` keyword in this same fashion for object references in general):

```
String s = null;
```

The plus sign (+) operator is normally used for arithmetic addition, but when used in conjunction with `Strings`, it represents **string concatenation**. Any number of `String` values can be concatenated with the + operator, as the following code snippet illustrates:

```
String x = "foo";
String y = "bar";
String z = x + y + "!"; // z assumes the value "foobar!" (x and y's values are
                        // unaffected)
```

You’ll learn about some of the many other operations that can be performed with or on `Strings`, along with insights into their OO nature, in Chapter 13.

Case Sensitivity

Java is a **case-sensitive** language. That is, the use of uppercase versus lowercase in Java is deliberate and mandatory, for example:

- Variable names that are spelled the same way but that differ in their use of case represent *different* variables:

```
// These are two DIFFERENT variables as far as the Java compiler
// is concerned.
int x; // lowercase
int X; // uppercase
```

- All keywords are rendered in lowercase: `public`, `class`, `int`, `boolean`, and so forth. ***Don't get “creative” about capitalizing these***, as the compiler will violently object—often with unintelligible compilation error messages, as in the following example, where the reserved word `for` is improperly capitalized:

```
// The reserved word 'for' should be lowercase.
For (int i = 0; i < 3; i++) {
```

which in turn produces the following seemingly bizarre compiler error:

```
' .class' expected
For (int i = 0; i < 3; i++) {
    ^
```

- The name of the `main` method is lowercase.
- As mentioned earlier, the `String` type starts with an uppercase “S.”

Java Expressions

Java is an **expression-oriented language**. A **simple expression** in Java is either

- A constant: `7`, `false`
- A `char`(acter) literal enclosed in single quotes: `'A'`, `'3'`
- A `String` literal enclosed in double quotes: `"foo"`, `"Java"`
- The name of any properly declared variables: `myString`, `x`
- Any **two** of the preceding types of expression that are combined with one of the Java **binary operators** (discussed in detail later in this chapter): `x + 2`
- Any **one** of the preceding types of expression that is modified by one of the Java **unary operators** (discussed in detail later in this chapter): `i++`
- Any of the preceding types of expression enclosed in parentheses: `(x + 2)`

plus a few more types of expression having to do with objects that you’ll learn about later in the book.

Expressions of arbitrary complexity can be assembled from the various different simple expression types by nesting parentheses, for example: `((((4/x) + y) * 7) + z)`.

Arithmetic Operators

The Java language provides a number of basic arithmetic operators, as shown in Table 2-1.

Table 2-1. *Java Arithmetic Operators*

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder (the remainder when the operand to the left of the % operator is divided by the operand to the right; e.g., <code>10 % 3 = 1</code> , because 3 goes into 10 three times, leaving a remainder of 1)

The `+` and `-` operators can also be used as unary operators to indicate positive or negative numbers: `-3.7`, `+42`.

In addition to the simple assignment operator, =, there are a number of specialized **compound assignment operators**, which combine variable assignment with an arithmetic operation, as shown in Table 2-2.

Table 2-2. *Java Compound Assignment Operators*

Operator	Description
+=	a += b is equivalent to a = a + b.
-=	a -= b is equivalent to a = a - b.
*=	a *= b is equivalent to a = a * b.
/=	a /= b is equivalent to a = a / b.
%=	a %= b is equivalent to a = a % b.

The final two arithmetic operators that we'll look at are the **unary increment** (++) and **decrement** (--) operators, which are used to increase or decrease the value of an int variable by 1 or of a floating-point (float, double) value by 1.0. They're known as unary operators because they're applied to a single variable, whereas binary operators combine the values of two expressions as discussed previously.

The unary increment and decrement operators can also be applied to char variables to step forward or backward one character position in the Unicode sorting sequence. For example, in the following code snippet, the value of variable c will be incremented from 'e' to 'f':

```
char c = 'e';
c++; // c will be incremented from 'e' to 'f'.
```

The increment and decrement operators can be used in either a **prefix** or **postfix** manner. If the operator is placed **before** the variable it's operating on (**prefix** mode), the increment or decrement of that variable is performed **before** the variable's **updated** value is used in any assignments made via that statement. For example, consider the following code snippet, which uses the prefix increment (++) operator. Assume that a and b have previously been declared as int variables in our program:

```
a = 1;
b = ++a;
```

After the preceding lines of code have executed, the value of variable a will be 2, as will the value of variable b. This is because, in the second line of code, the increment of variable a (from 1 to 2) occurs **before** the value of a is assigned to variable b. Thus, the single line of code

```
b = ++a;
```

is logically equivalent to the following **two** lines of code:

```
a = a + 1; // Increment a's value first ...
b = a;     // ... THEN use its value.
```

On the other hand, if the increment/decrement operator is placed **after** the variable it's operating on (**postfix** mode), the increment or decrement occurs **after** the variable's **original** value is used in any assignments made via that statement. Let's look at the same code snippet with the increment operator written in a **postfix** manner:

```
a = 1;
b = a++;
```

After the preceding lines of code have executed, the value of variable *b* will be 1, whereas the value of variable *a* will be 2. This is because, in the second line of code, the increment of variable *a* (from 1 to 2) occurs *after* the value of *a* is assigned to variable *b*. Thus, the single line of code

```
b = a++;
```

is logically equivalent to the following *two* lines of code:

```
b = a;      // Use a's value first ...
a = a + 1;  // ... THEN increment its value.
```

Here's a slightly more complex example; please read the accompanying comment to make sure that you can see how *x* will end up being assigned the value 10:

```
int y = 2;
int z = 4;
int x = y++ * ++z; // x will be assigned the value 10, because z will be
                  // incremented from 4 to 5 BEFORE its value is used in the
                  // multiplication expression, whereas y will remain at 2 until
                  // AFTER its value is used in the multiplication expression.
```

As you'll see in a bit, the increment and decrement operators are commonly used in conjunction with loops.

Relational and Logical Operators

A **logical expression** compares two (simple or complex) expressions *exp1* and *exp2* in a specified way, resolving to a boolean value of true or false.

To create logical expressions, Java provides the **relational operators** shown in Table 2-3.

Table 2-3. *Java Relational Operators*

Operator	Description
<i>exp1</i> == <i>exp2</i>	true if <i>exp1</i> equals <i>exp2</i> (note use of a double equal sign for testing equality).
<i>exp1</i> > <i>exp2</i>	true if <i>exp1</i> is greater than <i>exp2</i> .
<i>exp1</i> >= <i>exp2</i>	true if <i>exp1</i> is greater or equal to <i>exp2</i> .
<i>exp1</i> < <i>exp2</i>	true if <i>exp1</i> is less than <i>exp2</i> .
<i>exp1</i> <= <i>exp2</i>	true if <i>exp1</i> is less than or equal to <i>exp2</i> .
<i>exp1</i> != <i>exp2</i>	true if <i>exp1</i> is not equal to <i>exp2</i> (! is read as “not”).
! <i>exp</i>	true if <i>exp</i> is false, and false if <i>exp</i> is true.

In addition to the relational operators, Java provides **logical operators** that can be used to combine/modify logical expressions. The most commonly used logical operators are listed in Table 2-4.

Table 2-4. *Java Logical Operators*

Operator	Description
<i>exp1</i> && <i>exp2</i>	Logical “and”; compound expression is true only if both <i>exp1</i> and <i>exp2</i> are true
<i>exp1</i> <i>exp2</i>	Logical “or”; compound expression is true if either <i>exp1</i> or <i>exp2</i> is true
! <i>exp</i>	Logical “not”; toggles the value of a logical expression from true to false and vice versa

Here’s an example that uses the logical “and” operator to program the compound logical expression “if x is greater than 2.0 and y is not equal to 4.0”:

```
if ((x > 2.0) && (y != 4.0)) { ... }
```

Logical expressions are most commonly seen used with flow-control structures, discussed later in this chapter.

Evaluating Expressions and Operator Precedence

As mentioned earlier in the chapter, expressions of arbitrary complexity can be built up by layering nested parentheses—for example, $((8 * (y + z)) + y) * x$. The compiler generally evaluates such expressions from the innermost to outermost parentheses, left to right. Assuming that x, y, and z are declared and initialized as shown here

```
int x = 1;
int y = 2;
int z = 3;
```

then the expression on the right-hand side of the following assignment statement

```
int answer = ((8 * (y + z)) + y) * x;
```

would be evaluated piece by piece as follows:

$$\begin{array}{r}
 ((8 * (y + z)) + y) * x \\
 ((8 * 5) + y) * x \\
 (40 + y) * x \\
 42 * x \\
 42
 \end{array}$$

In the absence of parentheses, certain operators take precedence over others in terms of when they will be applied in evaluating an expression. For example, multiplication or division is performed before addition or subtraction. Operator precedence can be explicitly altered through the use of parentheses; operations performed inside parentheses take precedence over operations outside of parentheses. Consider the following code snippet:

```
int j = 2 + 3 * 4;    // j will be assigned the value 14
int k = (2 + 3) * 4; // k will be assigned the value 20
```

In the first line of code, which uses no parentheses, the multiplication operation takes precedence over the addition operation, and so the overall expression evaluates to the value $2 + 12 = 14$; it’s as if we’ve explicitly written $2 + (3 * 4)$ without having to do so.

In the second line of code, parentheses are explicitly placed around the operation $2 + 3$ so that the addition operation will be performed first, and the resultant sum will then be multiplied by 4 for an overall expression value of $5 * 4 = 20$.

Returning to an earlier example

```
if ((x > 2.0) && (y != 4.0)) { ... }
```

note that the $>$ and $!=$ operators take precedence over the $\&\&$ operator, such that we could eliminate the nested parentheses as follows:

```
if (x > 2.0 && y != 4.0) { ... }
```

However, the extra parentheses certainly don't hurt, and in fact it can be argued that they make the expression's intention clearer.

The Type of an Expression

The **type of an expression** is the Java type of the value to which the expression ultimately evaluates. For example, given the code snippet

```
double x = 3.0;
double y = 2.0;
if ((x > 2.0) && (y != 4.0)) { ... }
```

the expression $(x > 2.0) \&\& (y != 4.0)$ evaluates to true, and hence the expression $(x > 2.0) \&\& (y != 4.0)$ is said to be a boolean type expression. And, in the following code snippet

```
int x = 1;
int y = 2;
int z = 3;
int answer = ((8 * (y + z)) + y) * x;
```

the expression $((8 * (y + z)) + y) * x$ evaluates to 42, and hence the expression $((8 * (y + z)) + y) * x$ is said to be an `int`(eger) type expression.

Automatic Type Conversions and Explicit Casting

Java supports **automatic type conversion**. This means that if we try to assign a value to a variable

```
// Pseudocode.
x = expression;
```

and the expression on the *right-hand side* of the assignment statement evaluates to a *different* type than the type with which the variable on the *left-hand side* of the assignment statement was declared, Java will automatically convert the value of the right-hand expression to match the type of `x`, but *only if precision won't be lost in doing so*. This is best understood by looking at an example:

```
int x;
double y;
y = 2.7;
x = y; // We're trying to assign a double value to an int variable.
```

In the preceding code snippet, we're attempting to assign the double value of *y*, 2.7, to *x*, which is declared to be an *int*. If this assignment were to take place, the fractional part of *y* would be truncated, and *x* would wind up with an integer value of 2. This represents a loss in precision, also known as a **narrowing conversion**.

A C or C++ compiler will permit this assignment, silently truncating the value. Rather than assuming that this is what we intend to do, however, the Java compiler will generate an error on the last line of code as follows:

```
possible loss of precision
found: double
required: int
```

In order to signal to the Java compiler that we're willing to accept the loss of precision, we must perform an **explicit cast**—that is, we must precede the expression whose value is to be converted with the target type, enclosed in parentheses:

```
// Pseudocode.
x = (type) expression;
```

In other words, we'd have to rewrite the last line of the preceding example as follows in order for the Java compiler to permit the assignment of a more precise floating-point value to a less precise integer variable:

```
int x;
double y;
y = 2.7;
x = (int) y; // This will compile now, because we have explicitly
             // informed the compiler that we WANT a
             // narrowing conversion to occur.
```

Of course, if we were to **reverse** the direction of the assignment, assigning the *int* value of variable *x* to the double variable *y*, the Java compiler would have no problem with the assignment:

```
int x;
double y;
x = 2;
y = x; // Assign a less precise int value to a double variable that is capable of
       // handling more precision - this is fine as is.
```

In this particular case, we're assigning a value of **less** precision—2—to a variable capable of **more** precision; *y* will wind up with the value of 2.0. This is known as a **widening conversion**. Such conversions are performed automatically in Java, and need not be explicitly cast.

Note that there's an idiosyncrasy with regard to assigning constant values to variables of type *float* in Java; the following statement won't compile:

```
float y = 3.5; // This won't compile!
```

because a numeric constant value with a fractional component like 3.5 is automatically treated by Java as a more precise double value, and so the compiler will view this as a narrowing conversion and will refuse to carry out the assignment. To **force** such an assignment, we must either explicitly cast the floating-point constant into a *float*:

```
float y = (float) 3.5;    // This will compile, thanks to the cast.
```

or, alternatively, we can force the constant on the right-hand side of the assignment statement to be viewed by the Java compiler as a float by using the suffix `F`, as shown here:

```
float y = 3.5F;    // OK, because we're indicating that the constant is to be
                  // treated as a float, not as a double.
```

Yet another option is to simply declare `double` rather than `float` variables whenever we wish to represent floating-point numeric values in a program.

We'll typically use `doubles` instead of `floats` whenever we need to declare floating-point variables in our SRS application in Part 3 of the book, just to avoid these hassles of type conversion.

Expressions of type `char` can be converted to any other numeric type, as illustrated in this next example:

```
char c = 'a';

// Assigning a char value to a numeric variable transfers its
// ASCII numeric equivalent value.
int x = c;
float y = c;
double z = c;

System.out.println(x);
System.out.println(y);
System.out.println(z);
```

Here's the output:

```
97
97.0
97.0
```

The only Java type that can't be cast, either implicitly or explicitly, into another type is the `boolean` type.

You'll see other applications of casting, involving objects, later in the book.

Loops and Other Flow-Control Structures

Very rarely will a program execute sequentially, line by line, from start to finish. Instead, the path of execution through a program's logic will often be *conditional*.

- It may be necessary to have the program execute a certain block of code if some condition is met, or a *different* block of code if the condition *isn't* met.
- A program may have to repeatedly execute a particular block of code a fixed number of times, or until a particular result is attained.

The Java language provides a number of different types of loops and other flow-control structures to take care of these situations.

if Statements

The if statement is a basic conditional branch statement that executes one or more lines of code if a condition, represented as a logical expression, is satisfied. Alternatively, one or more lines of code can be executed if the condition is **not** satisfied by placing that code after the keyword else. The use of an else clause with an if statement is optional.

The basic syntax of the if statement is as follows:

```
// Pseudocode.  
if (logical-expression) {  
    execute whatever code is contained within these braces  
    if logical-expression evaluates to true  
}
```

Or, adding an optional else clause:

```
// Pseudocode.  
if (logical-expression) {  
    execute whatever code is contained within these braces  
    if logical-expression evaluates to true  
}  
else {  
    execute whatever code is contained within these braces if  
    logical-expression evaluates to false  
}
```

If only one executable statement follows either the if or (optional) else keyword, the braces can be omitted, as shown here:

```
// Pseudocode.  
if (logical-expression) single statement to execute if logical-expression is true;  
else single statement to execute if logical-expression is false;
```

For example:

```
if (x > 3) y = x;  
else z = x;
```

But it's generally considered good practice to always use braces as follows:

```
if (x > 3) {  
    y = x;  
}  
else {  
    z = x;  
}
```

A single boolean variable, as a simple form of Boolean expression, can serve as the logical expression/condition of an if statement. For example, it's perfectly acceptable to write the following:

```
// Use boolean variable "finished" as a flag that will get set to true when
// some particular operation is completed.
boolean finished;

// Initialize it to false.
finished = false;

// The details of intervening code, in which the flag may or may not get set to
// true, have been omitted ...

// Test the flag.
if (finished) { // equivalent to: if (finished == true) {
    System.out.println("We are finished! :o");
}
```

The ! (“not”) operator can be used to negate a logical expression, so that the block of code associated with an if statement is executed when the expression evaluates to false instead:

```
if (!finished) { // equivalent to: if (finished == false)
    System.out.println("We are NOT finished ... :op");
}
```

When testing for equality of two expressions, remember that we must use *two* consecutive equal signs, not just one:

```
if (x == 3) { // Note use of double equal signs (==) to test for equality.
    y = x;
}
```

A common mistake made by beginning Java programmers—particularly those who’ve previously programmed in C or C++—is to try to use a *single* equal sign to test for equality, as in this example:

```
// Note incorrect use of single equal sign below.
if (x = 3) {
    y = x;
}
```

In Java, an if test must be based on a valid *logical* expression; `x = 3` isn’t a logical expression, but rather an *assignment* expression. In fact, the preceding if statement won’t even compile in Java, whereas it *would* compile in the C and C++ programming languages, because in those languages, if tests are based on evaluating expressions to either the *integer* value 0 (interpreted to mean false) or nonzero (interpreted to mean true).

It’s possible to nest if-else constructs to test more than one condition. If nested, an inner if (plus optional else) statement is placed within the else part of an outer if. The basic syntax for a two-level nested if-else construct is shown here: